



对外经济贸易大学

University of International Business and Economics

成绩

2024 — 2025 学年第二学期期中考试

时间序列分析课程报告

报告题目：时间序列模型在谷物产量和绵羊数量预测中的应用

课程代码：STA308

课 序 号：3

课程名称：时间序列分析

学 号：

姓 名：6星

学 院：统计学院

专 业：统计学

考试时间：2025 年 6 月

对外经济贸易大学

本科课程期末论文或其他方式考试评阅表

请选择总分计算方式：1. 总分为各项指标平均分（ ） 2. 总分为各项指标分总和（ ）

指标项*	选题	观点	材料	文字水平	格式与框架	总分
分数						

论文评语：此页用黑笔填写，论文中用红笔批改。

任课教师签字：

年 月 日

教务处制表

摘要

本文深入探讨了时间序列分析在实际数据中的应用，通过对某地区连续 74 年谷物产量和 1867-1938 年英国绵羊数量两个时间序列数据的详细分析，系统展示了时间序列分析的完整流程。

首先，本文对谷物产量数据进行了序列平稳性与随机性的判定，运用时序图、ACF 图和 PACF 图进行初步诊断，结合 ADF 和 KPSS 检验验证了序列的平稳性。进一步，通过纯随机性检验确认序列非白噪声特性，表明序列中蕴含可提取的信息。在模型拟合阶段，基于 ACF 和 PACF 图特征，筛选出 AR(1)、MA(2) 和 ARMA 系列模型作为备选，综合 AIC 和 BIC 指标确定 **ARMA(1,1)** 模型最优，该模型参数显著，拟合效果最佳。利用选定模型对未来 5 年谷物产量进行预测，得出年均增长趋势及相应置信区间，为产量预估提供科学依据。

针对英国绵羊数量数据，本文分析流程类似。初始诊断显示原始序列具非平稳特性，经一阶差分处理后序列平稳化。差分后序列的 ACF 和 PACF 图提示 **AR(3)** 模型适配，模型拟合结果表明参数大多显著，残差检验确认模型拟合优良，能有效捕捉绵羊数量波动趋势。

本文研究彰显了时间序列分析方法在农业与畜牧业数据挖掘中的有效性，不仅精准捕捉数据波动特征，还为未来趋势预测提供有力支撑，对相关领域决策规划具重要参考价值。

关键词：时间序列分析；ARMA 模型；ARIMA 模型；谷物产量；英国绵羊数量；平稳性检验；一阶差分

目录

摘要.....	I
表格与插图清单.....	III
一、某地区连续 74 年的谷物产量时序分析.....	1
(一) 序列平稳性与随机性的判定.....	1
1. 序列平稳性初步判定.....	1
2. ADF 与 KPSS 平稳性检验.....	2
3. 序列纯随机性检验.....	4
(二) 模型拟合、定阶与检验.....	4
1. 备选模型 1: AR(1) 模型.....	5
2. 备选模型 2: MA(2) 模型.....	5
3. 备选模型 3: ARMA 系列模型.....	5
4. 模型拟合.....	5
5. 模型定阶与选择.....	7
6. 模型的显著性检验.....	7
7. 参数的显著性检验.....	9
8. 该地区未来 5 年的谷物产量.....	11
二、1867-1938 年英国绵羊数量时序分析.....	12
(一) 序列平稳性的判定.....	12
1. 原始序列平稳性初步判定.....	12
2. 原始序列的 ADF 检验.....	13
3. 一阶差分序列的平稳性初步判定.....	14
4. 一阶差分序列的 ADF 检验.....	15
5. 一阶差分序列的纯随机性检验.....	16
(二) 模型拟合、定阶与检验.....	17
1. 备选模型 1: AR(3) 模型.....	17
2. 备选模型 2: MA(1), MA(4) 模型.....	17
3. 备选模型 3: ARMA 系列模型.....	17
4. 模型定阶与选择.....	17
5. 模型拟合.....	18
6. 模型检验.....	19
7. 参数检验与模型优化.....	21
8. 拟合预测英国 1867-1938 年的绵羊数量.....	21
参考文献.....	23
附录.....	24

表格与插图清单

图 1 某地区连续 74 年的谷物产量时序图.....	1
图 2 谷物产量序列的 ACF 图（左）与 PACF 图（右）	1
图 3 谷物产量残差序列图	8
图 4 谷物产量残差的 ACF 图（左）和 PACF 图（右）	8
图 5 谷物产量残差的 Q-Q 图（左）和密度图（右）	8
图 6 谷物产量残差序列 P 值显著性图	9
图 7 谷物产量时间序列与预测图	12
图 8 1867-1938 年英国绵羊数量时间序列图	12
图 9 绵羊数量序列的 ACF 图（左）和 PACF 图（右）	13
图 10 一阶差分序列图	14
图 11 一阶差分序列的 ACF 图（左）与 PACF 图（右）	15
图 12 一阶差分序列的残差图	19
图 13 一阶差分序列残差 ACF 图（左）和 PACF 图（右）	20
图 14 一阶差分序列残差的 Q-Q 图（左）和密度图（右）	20
图 15 英国绵羊数量时间序列与预测图	22
表 1 三种自回归类型的 ADF 检验结果（谷物产量序列）	3
表 2 KPSS 检验结果表	4
表 3 谷物产量序列的纯随机检验结果表	4
表 4 备选模型的参数估计表	5
表 5 各模型关键指标对比表	7
表 6 ARMA(1,1) 模型的参数估计	10
表 7 参数估计的显著性水平	11
表 8 未来 5 年谷物预测的预测值与预测区间	12
表 9 三种自回归类型的 ADF 检验结果（绵羊数量序列）	14
表 10 三种自回归类型的 ADF 检验结果（一阶差分序列）	16
表 11 一阶差分序列的纯随机性结果表	16
表 12 各模型关键指标对比表	18
表 13 AR(3) 参数估计表	19
表 14 一阶差分序列的残差序列 P 值显著性图	21
表 15 参数估计的显著性水平	21

时间序列模型在谷物产量和绵羊数量预测中的应用

一、某地区连续 74 年的谷物产量时序分析

(一) 序列平稳性与随机性的判定

1. 序列平稳性初步判定

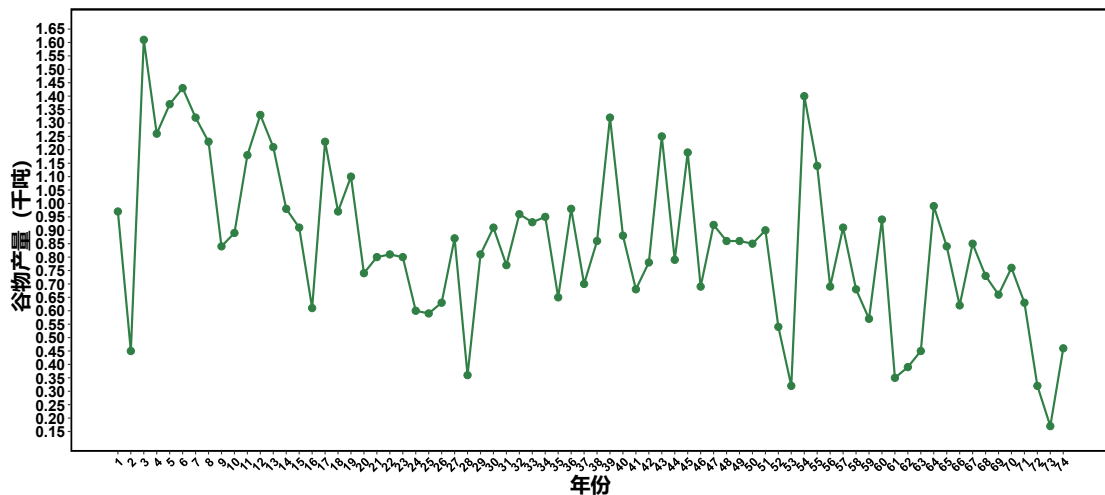


图 1 某地区连续 74 年的谷物产量时序图

图 1 展示了该时间序列的波动特征。从趋势分析来看，序列整体未呈现显著的上升或下降趋势，数据点围绕 0.75（千吨）随机波动，表明该序列可能不存在明显的趋势性成分，这在一定程度上符合平稳性的要求。进一步观察波动模式，未发现规律的周期性波动，因而可以推测该序列不包含季节性成分，这同样体现了其平稳性特征。在方差稳定性方面，序列的波动幅度在不同时间段大致保持一致，没有出现显著的扩大或缩小，这说明该序列的方差相对稳定，是时间序列平稳性的重要判断依据之一。此外，序列虽存在少量极值点，但这些极值点的出现是随机的，未呈现出明显的聚集性或规律性，极有可能属于随机扰动所致，未对序列的整体平稳性造成显著影响。因此，可以初步判定该序列平稳，但还需进一步借助统计检验方法加以验证。

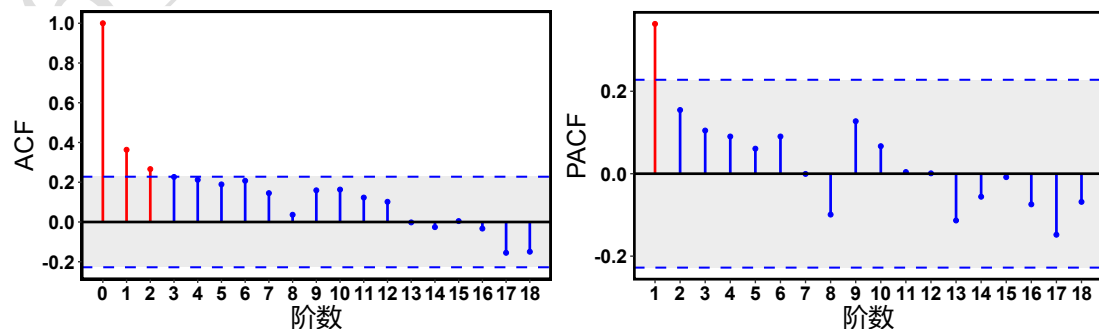


图 2 谷物产量序列的 ACF 图（左）与 PACF 图（右）

从图 2 的特征来看，该序列的平稳性特征较为显著。在 ACF 图中，自相关系数具有短期相关性，在 2 阶后呈现快速衰减趋势，并未能呈现明显的周期性或长期趋势，而这一特点恰与平稳序列的典型特征相吻合。在 PACF 图中，偏自相关系数在延迟阶数 1 之后同样展现出明显的衰减趋势。基于 ACF 图与 PACF 图的观察结果，二者均有力地支持了序列的平稳性。

2. ADF 与 KPSS 平稳性检验

为确保结论的严谨性，接下来，本文将结合 ADF 检验方法对该序列的平稳性进行进一步验证。

由图 1 可知，该序列不具有明显的趋势性；由图 2 可知，该序列的 ACF 和 PACF 的延迟阶数均小于等于 2。考虑到人为判断的主观性，本文选择无趋势情况下的无漂移项自回归结构（类型 1）、有漂移项自回归结构（类型 2）和有趋势情况下的有偏移的回归结构（类型 3）进行延迟 0-2 阶的 ADF 检验。其检验步骤如下所示：

假设该序列 $\{x_t\}$ 的确定性部分可以由过去 p 期的历史数据描述，即序列可以表达为

$$x_t = \phi_1 x_{t-1} + \phi_2 x_{t-2} + \cdots + \phi_p x_{t-p} + \varepsilon_t \quad (1)$$

其可以等价表达为

$$\nabla x_t = \rho x_{t-1} - \beta_1 \nabla x_{t-1} - \cdots - \beta_{p-2} \nabla x_{t-p+1} - \beta_{p-1} \nabla x_{t-p+1} + \xi_t \quad (2)$$

其中 $\rho = \phi_1 + \phi_2 + \cdots + \phi_p - 1$ ， $\beta_j = \phi_{j+1} + \phi_{j+2} + \cdots + \phi_p$

【建立假设】

$$H_0: \rho = 0 \Leftrightarrow H_1: \rho < 0$$

【构建检验统计量 ADF】

$$\tau = \frac{\hat{\rho}}{S(\hat{\rho})}, \text{ 其中 } S(\hat{\rho}) \text{ 为 } \hat{\rho} \text{ 的样本标准差}$$

【代入数据求 P 值】

ADF 检验的类型 1（无漂移、无趋势的自回归结构）为

$$x_t = \phi_1 x_{t-1} + \phi_2 x_{t-2} + \cdots + \phi_p x_{t-p} + \xi_t \quad (3)$$

ADF 检验的类型 2（有漂移、无趋势的自回归结构）为

$$x_t = \phi_0 + \phi_1 x_{t-1} + \phi_2 x_{t-2} + \cdots + \phi_p x_{t-p} + \xi_t \quad (4)$$

ADF 检验的类型 3（有漂移、有趋势的回归结构）为

$$x_t = \phi_0 + \beta t + \phi_1 x_{t-1} + \phi_2 x_{t-2} + \cdots + \phi_p x_{t-p} + \xi_t \quad (5)$$

利用 R 语言，针对三种回归模型（类型 1、类型 2 和类型 3）求解 ADF 检验的 P 值，结果如表 1 所示。

表 1 三种自回归类型的 ADF 检验结果（谷物产量序列）

检验类型	延迟阶数	检验统计量 ADF 值	P 值	结论 ($\alpha=0.05$)
无漂移无趋势项	0	-1.7253	0.08304*	非平稳
无漂移无趋势项	1	-1.0322	0.2838	非平稳
无漂移无趋势项	2	-1.2549	0.2129	非平稳
有漂移无趋势项	0	-5.6238	$\leq 0.01^{***}$	平稳
有漂移无趋势项	1	-3.6566	$\leq 0.01^{***}$	平稳
有漂移无趋势项	2	-2.6829	0.08564*	非平稳
有漂移和趋势项	0	-7.1173	$\leq 0.01^{***}$	平稳
有漂移和趋势项	1	-5.5609	$\leq 0.01^{***}$	平稳
有漂移和趋势项	2	-3.9794	0.01543**	平稳

注：*表示显著性 P 值小于 0.10，**表示显著性 P 值小于 0.05，***表示显著性 P 值小于 0.01

由表 1 可知，在显著性水平 $\alpha=0.05$ 的情况下，当不考虑偏移项时，无论延迟阶数如何，序列均表现出非平稳特性。当考虑偏移项时，延迟阶数为 0 和 1 时，序列表现出平稳性；但延迟阶数为 2 时，序列又表现出非平稳性。当考虑漂移项和趋势项时，该序列在 0-2 阶可以认为是平稳的。王燕^[1]认为在对多种模型进行 ADF 检验后，当某种类型检验存在多阶延迟均平稳时，可以判定该序列为平稳序列。因此，当考虑漂移项和趋势项时，可以判断该序列为平稳性序列。由于 ADF 检验结果对延迟阶数和模型类型较为敏感，本文将进行 KPSS 检验进一步验证平稳性。

KPSS 检验（Kwiatkowski-Phillips-Schmidt-Shin 检验）是一种用于检验时间序列平稳性的统计方法。其核心思想是假设序列是平稳的，并通过检验来判断是否拒绝这一假设。

【建立假设】

H_0 : 序列平稳或趋势平稳 $\Leftrightarrow H_1$: 序列非平稳

【构建检验统计量 KPSS】

$$KPSS = \frac{\frac{1}{T} \sum_{t=1}^T \left(\sum_{i=1}^t \hat{e}_i \right)^2}{\hat{\sigma}^2}$$
，其中 T 为样本容量， $\hat{e}_i$ 是序列的残差， $\hat{\sigma}^2$ 是残

差的方差估计。

【代入数据求 P 值】

KPSS 检验的类型 1（无趋势的自回归结构）为

$$x_t = \phi_0 + \phi_1 x_{t-1} + \phi_2 x_{t-2} + \cdots + \phi_p x_{t-p} + \xi_t \quad (6)$$

KPSS 检验的类型 2（有趋势的自回归结构）为

$$x_t = \alpha + \beta t + \phi_1 x_{t-1} + \phi_2 x_{t-2} + \cdots + \phi_p x_{t-p} + \xi_t \quad (7)$$

利用 R 语言，针对两种自回归模型（类型 1 和类型 2）求解 KPSS 检验的 P 值，结果如表 2 所示。

表 2 KPSS 检验结果表

检验类型	检验统计量	P 值	结论 ($\alpha=0.05$)
无趋势	0.93126	$\leq 0.01^{***}$	非平稳
有趋势	0.10853	0.1	平稳

注：*表示显著性 P 值小于 0.10，**表示显著性 P 值小于 0.05，***表示显著性 P 值小于 0.01

由表 2 可知，无趋势 KPSS 检验统计量为 0.93126， P 值小于 0.01（实际 P 值更小），拒绝原假设，表明序列不是围绕常数均值的平稳序列；有趋势 KPSS 检验统计量为 0.10853， P 值为 0.1，大于 0.05，无法拒绝原假设，表明序列可能是围绕某个趋势的平稳序列。

综合上述两种检验结果，可以判断该序列平稳。

3. 序列纯随机性检验

【建立假设】

$$H_0: \rho_1 = \rho_2 = \cdots = \rho_m = 0, \forall m \geq 1$$

$$H_1: \text{至少存在某个 } \rho_k \neq 0, \forall m \geq 1, k \leq m$$

【构建检验统计量 Q 】

$$Q = n(n+2) \sum_{k=1}^m \left(\frac{\hat{\rho}_k^2}{n-k} \right), \text{ 其中 } n \text{ 为序列观测期数, } m \text{ 为延迟阶数, 且 } Q$$

近似服从 $\chi^2(m)$ 。

【代入数据求 P 值】

本文将该序列进行延迟 6 阶、12 阶和 18 阶的纯随机性检验，代入 R 得到结果如表 3 所示。

表 3 谷物产量序列的纯随机检验结果表

延迟阶数	检验统计量	P 值	结论 ($\alpha=0.05$)
6	27.983	0.00004156***	非白噪声
12	35.375	0.0001234***	非白噪声
18	38.941	0.0007095***	非白噪声

注：*表示显著性 P 值小于 0.10，**表示显著性 P 值小于 0.05，***表示显著性 P 值小于 0.01

由表 3 可知，延迟 6 阶、12 阶和 18 阶的 LB 统计量的 P 值均小于显著性水平 $\alpha = 0.01$ ，所以有理由拒绝该序列为纯随机性的原假设，即该序列为非白噪声序列。

（二）模型拟合、定阶与检验

由表 1 可知，当考虑漂移项和趋势项的回归结构时，可以判断该序列为平稳性序列，并且通过了纯随机性检验，可以进一步采用平稳模型进行建模分析。根据图 2 的结果知存在三种备选模型。

1. 备选模型 1: AR(1) 模型

由 ACF 图可以看出, 自相关系数是以一种有规律按指数函数轨迹递减的, 是典型的拖尾特征; 由 PACF 图可以看出, 除了 1 阶偏自相关系数在 2 倍标准差以外, 其他阶数的偏自相关系数均在 2 倍标准差范围内, 这是偏自相关系数 1 阶截尾的典型特征, 可以将 **AR(1) 模型** 作为备选模型。

2. 备选模型 2: MA(2) 模型

由 ACF 图可以看出, 除了延迟 1 阶和 2 阶的自相关系数在 2 倍标准差以外, 其他阶数的自相关系数均在 2 倍标准差范围内波动, 可以认为该序列自相关系数 2 阶截尾; 由 PACF 图可以看出, 偏自相关系数显示出拖尾的典型特征, 可以将 **MA(2) 模型** 作为备选模型。

3. 备选模型 3: ARMA 系列模型

由 ACF 和 PACF 图可以看出, 自相关系数和偏自相关系数均是以一种有规律按指数函数轨迹递减的, 是典型的拖尾特征, 且都显示出低阶延迟在 2 倍标准差以外, 因此, 可以将 **ARMA(1,1)**, **ARMA(1,2)**, **ARMA(2,1)** 和 **ARMA(2,2)** 作为备选模型。

4. 模型拟合

本文利用 R 语言对该序列用 3 种备选模型进行拟合, 并采用 CSS-MLE 混合法对模型的参数进行估计, 最终得到的结果如 **表 4** 所示。

表 4 备选模型的参数估计表

备选模型	ar1 (标准误差)	ar2 (标准误差)	ma1 (标准误差)	ma2 (标准误差)	截距 (标准误差)
AR(1)	0.3681 (0.1085)	-	-	-	0.8491 (0.0499)
MA(2)	-	-	0.2894 (0.1169)	0.1734 (0.1093)	0.8488 (0.0464)
ARMA(1,1)	0.9355 (0.0837)	-	-0.7266 (0.1475)	-	0.8319 (0.1135)
ARMA(1,2)	0.9462 (0.0736)	-	-0.6959 (0.1426)	-0.0593 (0.1233)	0.8352 (0.1187)
ARMA(2,1)	1.0174 (0.1869)	-0.0678 (0.1489)	-0.7715 (0.1483)	-	0.8346 (0.1184)
ARMA(2,2)	0.6334 (1.2623)	0.2952 (1.1844)	-0.3830 (1.2411)	-0.2912 (0.9013)	0.835 (0.119)

【AR(1) 模型】

在 AR(1) 模型中, ar1 估计为 0.3681, 标准误差 0.1085, 显著高于两倍标准

误差 0.2170。截距项估计为 0.8491，标准误差 0.0499，同样显著高于两倍标准误差 0.0998。表明 AR(1) 模型参数整体显著，拟合效果可靠。

【MA(2) 模型】

对于 MA(2) 模型，ma1 估计为 0.2894，标准误差 0.1169，超过两倍标准误差 0.2338，具有显著性。然而，ma2 估计为 0.1734，标准误差 0.1093，未达两倍标准误差 0.2186，不显著。截距项估计为 0.8488，标准误差 0.0464，显著高于两倍标准误差 0.0928。表明 MA(2) 模型中 ma1 和截距显著，但 ma2 不显著，模型拟合效果受限。

【ARMA(1,1) 模型】

在 ARMA(1,1) 模型里，ar1 估计为 0.9355，标准误差 0.0837，远超两倍标准误差 0.1674。ma1 估计为 -0.7266，标准误差 0.1475，绝对值显著高于两倍标准误差 0.2950。截距项估计为 0.8319，标准误差 0.1135，超过两倍标准误差 (0.2270)。所有参数均显著，ARMA(1,1) 模型拟合效果优异。

【ARMA(1,2) 模型】

在 ARMA(1,2) 模型中，ar1 估计为 0.9462，标准误差 0.0736，超过两倍标准误差 0.1472。ma1 估计为 -0.6959，标准误差 0.1426，绝对值高于两倍标准误差 0.2852。然而，ma2 估计为 -0.0593，标准误差 0.1233，未达两倍标准误差 0.2466。截距项估计为 0.8352，标准误差 0.1187，显著高于两倍标准误差 0.2374。表明 ar1、ma1 和截距显著，但 ma2 不显著，模型存在潜在过拟合风险。

【ARMA(2,1) 模型】

对于 ARMA(2,1) 模型，ar1 估计为 1.0174，标准误差 0.1869，超过两倍标准误差 0.3738。ar2 估计为 -0.0678，标准误差 0.1489，未达两倍标准误差 0.2978。ma1 估计为 -0.7715，标准误差 0.1483，绝对值高于两倍标准误差 0.2966。截距项估计为 0.8346，标准误差 0.1184，超过两倍标准误差 0.2368。说明 ar1、ma1 和截距显著，但 ar2 不显著，模型存在冗余参数风险。

【ARMA(2,2) 模型】

在 ARMA(2,2) 模型中，ar1 估计为 0.6334，标准误差 1.2623，未达两倍标准误差 2.5246。ar2 估计为 0.2952，标准误差 1.1844，同样不显著。ma1 估计为 -0.3830，标准误差 1.2411，绝对值未达两倍标准误差 2.4822。ma2 估计为 -0.2912，标准误差 0.9013，不显著。截距项估计为 0.835，标准误差 0.119，显著高于两倍标准误差 (0.238)。但由于其他关键参数不显著，ARMA(2,2) 模型预测能力受限。

综上所述，ARMA(1,1) 模型的所有参数均显著，说明其在拟合谷物产量序列时效果较好，稳定性强，可以初步确定为备选模型中的最优选择。

5. 模型定阶与选择

在对各备选模型进行拟合与参数估计后，本文依据关键指标（包括方差估计值、对数似然值、AIC 值及 BIC 值）进行定阶与选择，得到结果如表 5 所示。这些指标综合反映了模型的拟合优度、复杂度以及预测能力，对模型选择具有关键指导意义。

表 5 各模型关键指标对比表

模型名称	方差估计值	对数似然值	AIC 值	BIC 值
AR(1)	0.07467	-9.07	24.13862	31.05081
MA(2)	0.07509	-9.27	26.53246	35.74872
ARMA(1,1)	0.06903	-6.37	20.73728	29.95354
ARMA(1,2)	0.0688	-6.25	22.50697	34.02729
ARMA(2,1)	0.06883	-6.27	22.53291	34.05324
ARMA(2,2)	0.06874	-6.23	24.45550	38.27990

ARMA(1,1) 模型的 AIC 和 BIC 值均为最低，且所有参数均显著，表明其在拟合效果和模型复杂度之间取得了最佳平衡。虽然 ARMA(1,2) 和 ARMA(2,2) 模型的方差估计值略低于 ARMA(1,1) 模型，但它们的 AIC 和 BIC 值较高，且存在不显著参数，模型复杂度较高。因此，结合各模型的综合表现，ARMA(1,1) 模型被选为最优模型，能够为谷物产量的预测与分析提供可靠依据。最终模型的形式为

$$x_t = \phi_0 + \phi_1 x_{t-1} + \varepsilon_t - \theta_1 \varepsilon_{t-1} \quad (8)$$

6. 模型的显著性检验

在对各备选模型进行拟合与参数估计后，本文基于关键指标（AIC、BIC 等）选定 ARMA(1,1) 模型作为最优模型。然而，模型选择仅是时间序列分析的一个重要环节，模型检验同样不可或缺，它能验证模型的合理性和适用性，确保模型能准确捕捉序列特征并可靠地进行预测。接下来，本文将从残差自相关性检验和模型正态性检验两个关键维度出发，对该模型进行全面诊断。

【建立假设】

$$H_0: \rho = 0 \Leftrightarrow H_1: \rho < 0$$

【构建检验统计量 ADF】

$$\tau = \frac{\hat{\rho}}{S(\hat{\rho})}, \text{ 其中 } S(\hat{\rho}) \text{ 为 } \hat{\rho} \text{ 的样本标准差}$$

【代入数据求 P 值】

ADF 检验的类型 1（无漂移、无趋势的自回归结构）为

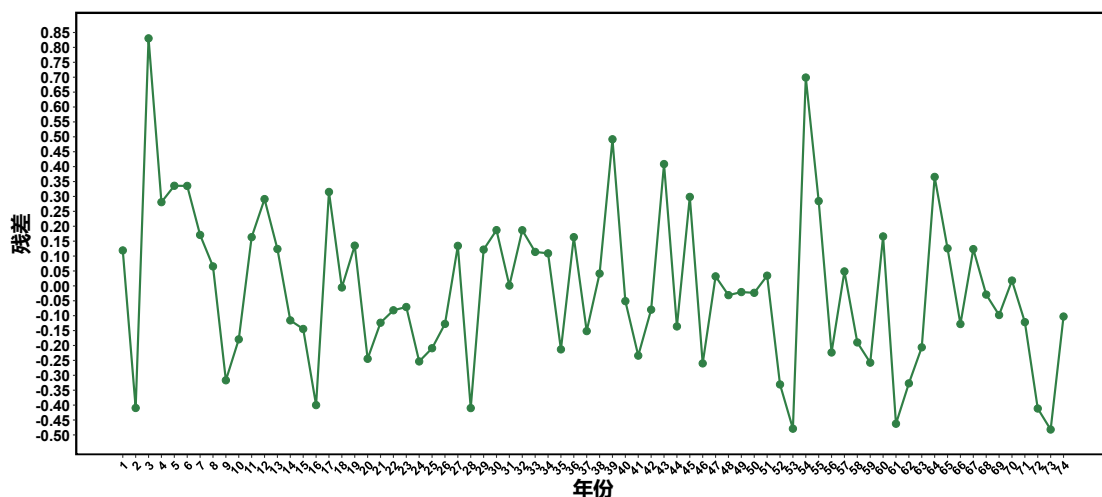


图 3 谷物产量残差序列图

图 3 显示残差在零附近随机波动，没有明显的趋势或周期性，这表明模型已经较好地捕捉了序列中的趋势和季节性成分，并且残差的波动幅度相对稳定，没有明显的增大或减小趋势，这表明模型的方差假设是合理的。

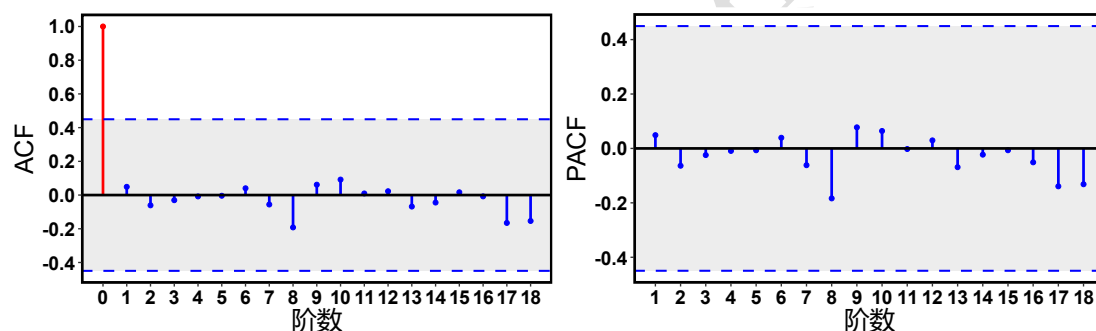


图 4 谷物产量残差的 ACF 图（左）和 PACF 图（右）

ACF 图显示，除了延迟阶数 0 外，其他延迟阶数的自相关系数都在置信区间内（蓝色虚线之间），这表明残差序列不存在显著的自相关性。这一结果支持了残差序列为白噪声的假设，说明模型已经充分提取了序列中的相关信息。PACF 图同样显示，除了延迟阶数 0 外，其他延迟阶数的偏自相关系数都在置信区间内，进一步支持了残差序列为白噪声的假设。

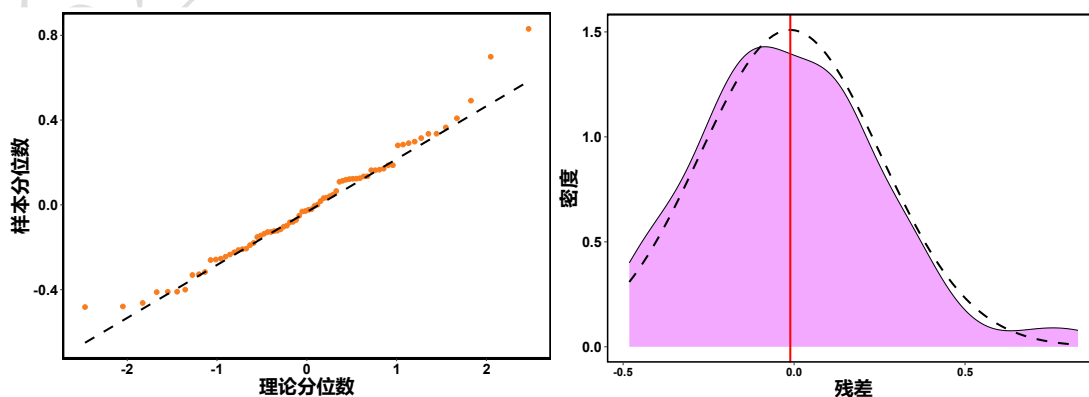


图 5 谷物产量残差的 Q-Q 图（左）和密度图（右）

Q-Q 图显示，残差点大致分布在一条直线上，这表明残差的分布接近正态分布。尽管在两端可能存在轻微的偏离，但整体上残差的正态性假设是合理的。密度图显示，残差的密度曲线与正态分布曲线（虚线）基本重合，进一步验证了残差的正态性假设。

在对模型残差进行正态性检验后，为进一步确保模型的适用性与可靠性，本文运用 Ljung-Box 检验对残差序列的自相关性进行严谨评估。Ljung-Box 检验作为一种广泛应用于时间序列分析的统计方法，能够有效检验序列在多个滞后阶数下的自相关性，其检验过程如下所示

【建立假设】

$$H_0: \rho_1 = \rho_2 = \cdots = \rho_m = 0, \forall m \geq 1$$

$$H_1: \text{至少存在某个 } \rho_k \neq 0, \forall m \geq 1, k \leq m$$

【构建检验统计量 ADF】

$$LB = n(n+2) \sum_{k=1}^m \frac{\hat{\rho}_k^2}{n-k} \sim \chi^2(m), \forall m > 0$$

【代入数据求 P 值】

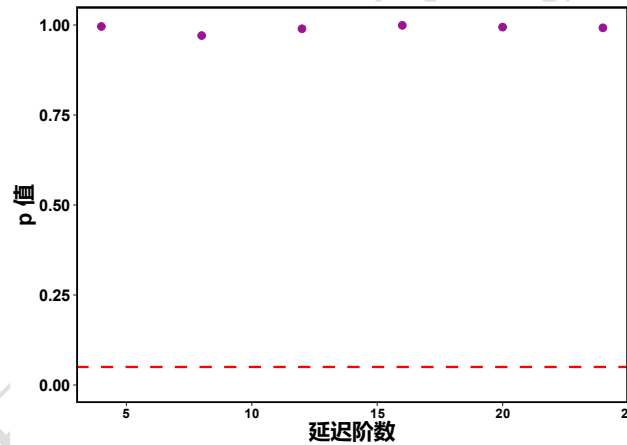


图 6 谷物产量残差序列 P 值显著性图

用 R 语言对 ARMA(1,1) 模型的残差进行 Ljung-Box 检验，结果显示在不同延迟阶数下，残差序列的 P 值均大于显著性水平 $\alpha = 0.05$ 。这些结果表明，残差序列可以视为白噪声，说明 ARMA(1,1) 模型充分提取了序列中的相关信息，拟合效果良好。

7. 参数的显著性检验

在完成了模型的拟合、定阶以及初步检验后，本文确定了 ARMA(1,1) 模型作为最优模型，并且通过了残差自相关性检验和正态性检验。接下来，本文将对模型中的各个参数进行进一步的检验，以确保这些参数的准确性和可靠性。

表 6 ARMA(1,1) 模型的参数估计

备选模型	ar1 (标准误差)	ma1 (标准误差)	截距 (标准误差)
ARMA(1,1)	0.9355 (0.0837)	-0.7266 (0.1475)	0.8319 (0.1135)

由表 6 可知, ϕ_1 估计为 0.9355, 标准误差 0.0837, 远超两倍标准误差 0.1674。 θ_1 估计为 -0.7266, 标准误差 0.1475, 绝对值显著高于两倍标准误差 0.2950。 ϕ_0 估计为 0.8319, 标准误差 0.1135, 超过两倍标准误差 (0.2270)。所有参数均显著, ARMA(1,1) 模型拟合效果优异。

为了进一步验证参数的显著性, 本文采取精确方法 t 检验验算其 P 值, 其步骤如下所示:

【建立假设】

$$H_0: \beta_j = 0 \Leftrightarrow H_1: \beta_j \neq 0, \forall 1 \leq j \leq m$$

【构建检验统计量 T 】

$$\text{易知 } E(\hat{\beta}) = E[(X'X)^{-1}X'\tilde{y}] = (X'X)^{-1}X'X\beta = \beta$$

$$\text{Var}(\hat{\beta}) = \text{Var}[(X'X)^{-1}X'\tilde{y}] = (X'X)^{-1}X'X(X'X)^{-1}\sigma_\varepsilon^2 = (X'X)^{-1}\sigma_\varepsilon^2$$

$$\Omega = (X'X)^{-1} = \begin{pmatrix} a_{11} & \cdots & a_{1m} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mm} \end{pmatrix}$$

在正态分布假定下, 第 j 个未知参数的最小二乘估计值 $\hat{\beta}_j$ 服从正态分布:
 $\hat{\beta}_j \sim N(0, a_{jj}\sigma_\varepsilon^2), \quad 1 \leq j \leq m$

由于 σ_ε^2 不可观测, 使用最小残差平方和估计 σ_ε^2 :

$$\hat{\sigma}_\varepsilon^2 = \frac{Q(\tilde{\beta})}{n-m}$$

根据正态分布性质, 有:

$$\frac{Q(\tilde{\beta})}{\sigma_\varepsilon^2} \sim \chi^2(n-m)$$

其中, n 为序列长度; m 为待估参数个数。

由此可以构造出用于检验未知参数显著性的 t 检验统计量:

$$T = \sqrt{n-m} \frac{\hat{\beta}}{\sqrt{a_{jj}Q(\tilde{\beta})}} \sim t(n-m)$$

当该检验统计量的绝对值大于自由度为 $n-m$ 的 t 分布的 $1-\alpha/2$ 分位点, 即

$$|T| \geq t_{1-\alpha/2}(n-m)$$

或者该检验统计量的 P 值小于 α 时，拒绝原假设，认为该参数显著非零。如果参数显著性检验不能拒绝原假设，就应剔除不显著参数，重新拟合结构更精练的模型。

根据 t 统计量的定义，参数估计值除以参数标准差即为该参数的 t 统计量：

$$t = \frac{\hat{\phi}_j}{\hat{\sigma}_j} \sim t(n-m)$$

$$\text{其中，} \hat{\sigma}_j = \sqrt{a_{jj}Q(\tilde{\beta})/(n-m)}$$

【代入数据求 P 值】

表 7 参数估计的显著性水平

参数	t 统计量	P 值
ϕ_1	11.175126	0.00000000001324125***
θ_1	4.925238	0.0000026514***
ϕ_0	7.332556	0.0000000001436201***

由表 7 可知，三个参数的 P 值均小于 0.001，拒绝原假设，即三个参数都显著非零。

因此，谷物序列最终的拟合模型为：

$$x_t = 0.8319 + 0.9355x_{t-1} + \varepsilon_t + 0.7266\varepsilon_{t-1} \quad (9)$$

8. 该地区未来 5 年的谷物产量

在 ARMA(p, q) 模型场合

$$x_t(l) = E(\phi_1 x_{t+l-1} + \cdots + \phi_p x_{t+l-p} + \varepsilon_{t+l} - \theta_1 \varepsilon_{t+l-1} - \cdots - \theta_q \varepsilon_{t+l-q} | x_t, x_{t-1}, \cdots)$$

$$= \begin{cases} \phi_1 \hat{x}_t(l-1) + \cdots + \phi_p \hat{x}_t(l-p) - \sum_{i=l}^q \theta_i \varepsilon_{t+l-i}, & l \leq q \\ \phi_1 \hat{x}_t(l-1) + \cdots + \phi_p \hat{x}_t(l-p), & l > q \end{cases}$$

式中，

$$\hat{x}_t(k) = \begin{cases} \hat{x}_t(k), & k \geq 1 \\ x_{t+k}, & k \leq 0 \end{cases}$$

预测方差为：

$$\text{Var}[e_t(l)] = (G_0^2 + G_1^2 + \cdots + G_{l-1}^2)\sigma_\varepsilon^2$$

利用 R 语言对未来 5 年的谷物进行预测，得到结果如表 8 和图 7 所示。

表 8 未来 5 年谷物预测的预测值与预测区间

年份	预测值	80%置信区间	95%置信区间
75	0.5587494	(0.2220444, 0.8954544)	(0.04380359, 1.073695)
76	0.5763729	(0.2323986, 0.9203471)	(0.05030958, 1.102436)
77	0.5928593	(0.2426472, 0.9430714)	(0.05725612, 1.128463)
78	0.6082821	(0.2527009, 0.9638633)	(0.06446755, 1.152097)
79	0.6227099	(0.2624956, 0.9829241)	(0.07180974, 1.173610)

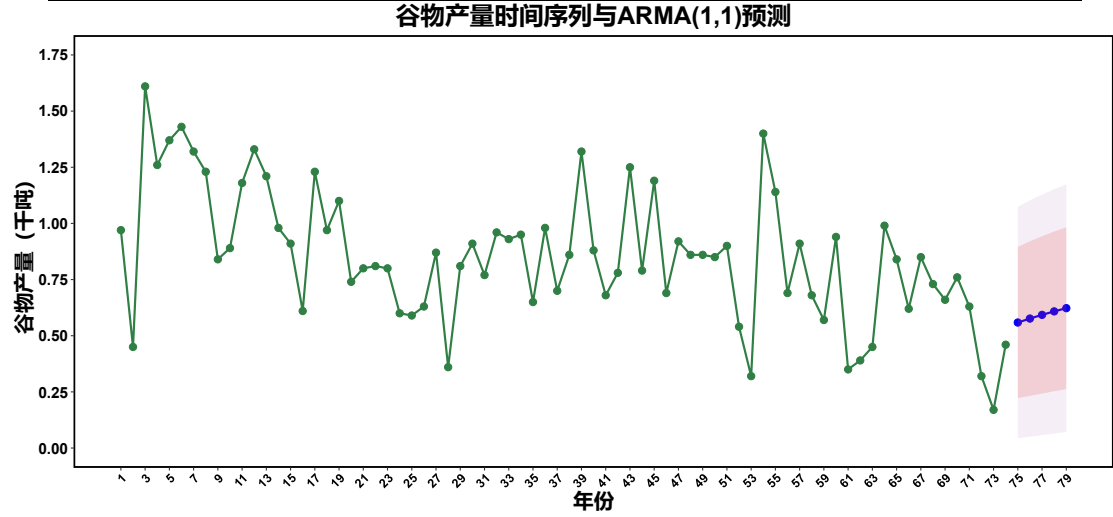


图 7 谷物产量时间序列与预测图

预测未来 5 年谷物产量呈现逐年递增趋势，年均增长约 0.018 千吨，产量从第 75 年的 0.5587 千吨逐步上升至第 79 年的 0.6227 千吨。预测结果存在一定的不确定性，80%置信区间和 95%置信区间宽度**逐渐增大**，表明随着预测年份增加，不确定性也在上升。未来 5 年该地区谷物产量有望实现持续增长。

二、1867-1938 年英国绵羊数量时序分析

(一) 序列平稳性的判定

1. 原始序列平稳性初步判定

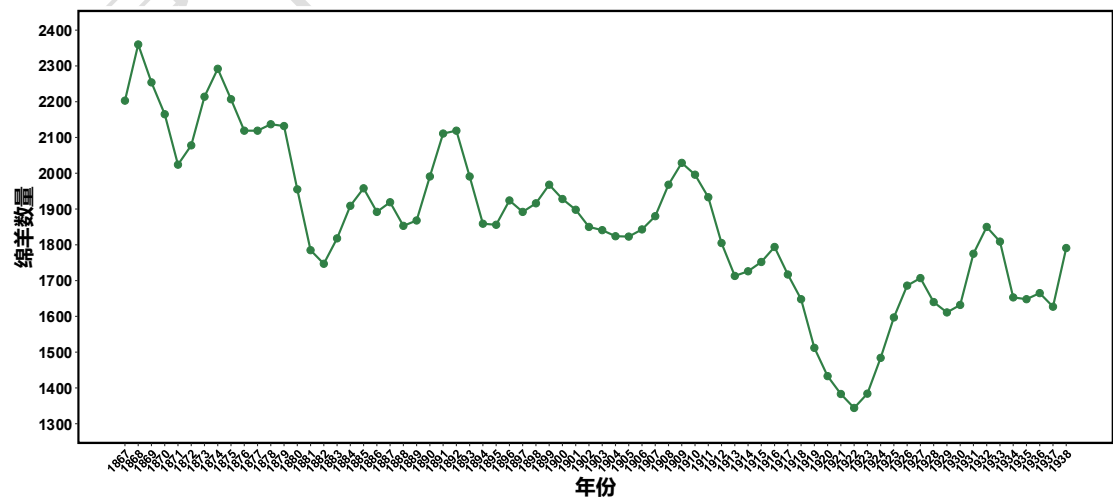


图 8 1867-1938 年英国绵羊数量时间序列图

图 8 展示了绵羊数量的历史时间序列变化情况。从图中可以看出，该序列存在明显的长期下降趋势，特别是在 20 世纪初期之后，绵羊数量整体呈现出逐步减少的态势。这表明该序列具有一定的**趋势性成分**，不符合严格意义上的平稳序列特征。进一步观察波动模式，可以发现在不同年份之间，绵羊数量的变化并不规律，未呈现出明确的周期性重复波动，说明该序列**不具有显著的季节性成分**。但需要注意的是，在部分年份（如 1919 至 1923 年）出现了连续的快速下降，可能受到外部历史事件或环境变化的影响。从方差稳定性来看，绵羊数量在早期（1867–1910 年）波动较大，而在后期（1920 年以后）波动幅度趋于收敛，说明**方差并不完全稳定**，这一变化可能影响序列的平稳性判断。此外，图中也存在部分异常点或剧烈变化的时段，例如 1920 年前后和 1930 年代中后期，绵羊数量出现了快速下跌或反弹，这些波动可能源于突发事件的冲击，具有较强的扰动性特征。综上所述，该序列存在明显的趋势成分，方差也有阶段性的变化，因此初步判断其**不满足平稳性要求**。

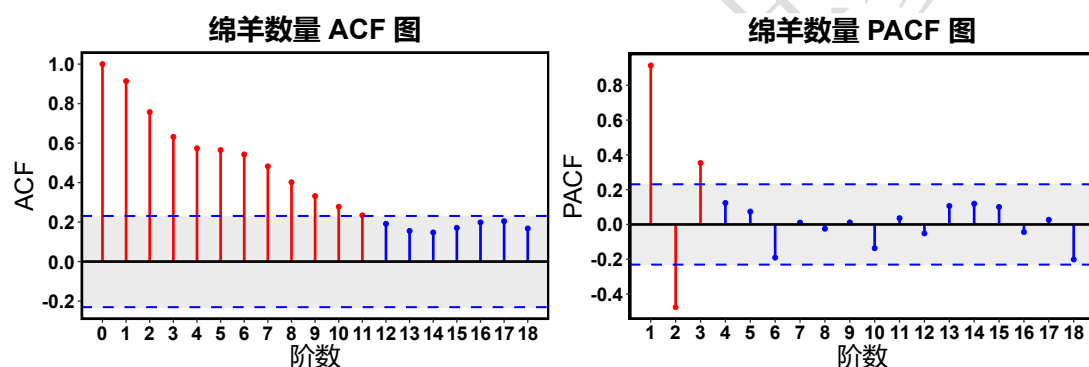


图 9 绵羊数量序列的 ACF 图（左）和 PACF 图（右）

从 ACF 图可以看出，自相关系数呈现出缓慢衰减的趋势，说明原始序列存在较强的趋势性，属于非平稳序列；而 PACF 图在延迟前 3 阶显著，其余阶数大致落在显著性范围内，表现出截尾特征，暗示该序列可能符合 AR(3) 模型结构。综合判断，该序列需要进行差分处理以实现平稳化，差分后可进一步分析 ACF 和 PACF 图来确定更合适的 ARIMA 模型参数。因此，可以优先考虑将该序列进行 1 阶差分后，建立低阶 AR 模型。

2. 原始序列的 ADF 检验

为了验证上述关于序列非平稳性的初步判定并进一步确认是否需要进行差分处理，本文将对原始序列进行 **ADF 检验**（该检验的检验过程见 一、（一）2）。

表 9 三种自回归类型的 ADF 检验结果（绵羊数量序列）

检验类型	延迟阶数	检验统计量 ADF 值	P 值	结论 ($\alpha=0.05$)
无漂移无趋势项	0	-0.8193	0.3514	非平稳
无漂移无趋势项	1	-0.9114	0.3221	非平稳
无漂移无趋势项	2	-0.6091	0.4183	非平稳
有漂移无趋势项	0	-1.9585	0.332	非平稳
有漂移无趋势项	1	-3.4083	0.01607**	平稳
有漂移无趋势项	2	-1.9898	0.3205	非平稳
有漂移和趋势项	0	-2.3222	0.4441	非平稳
有漂移和趋势项	1	-4.3014	≤ 0.01 ***	平稳
有漂移和趋势项	2	-2.4815	0.3793	非平稳

注：*表示显著性 P 值小于 0.10，**表示显著性 P 值小于 0.05，***表示显著性 P 值小于 0.01

由表 9 可知，在显著性水平 $\alpha=0.05$ 时，该序列无论采用何种检验类型和延迟阶数，整体呈现非平稳特性。具体来说，当无漂移无趋势项时，所有延迟阶数的 P 值均大于 0.05；即使引入漂移项或同时引入漂移项和趋势项，多数情况下的 P 值仍大于 0.05，仅个别延迟阶数（如延迟阶数为 1 时）的 P 值小于 0.05，但不足以改变整体非平稳的结论。因此，可以明确判断该序列为非平稳序列。

3. 一阶差分序列的平稳性初步判定

由于原始序列是非平稳序列，本文进行一阶差分运算后，得到一阶差分时序图如图 10 所示。

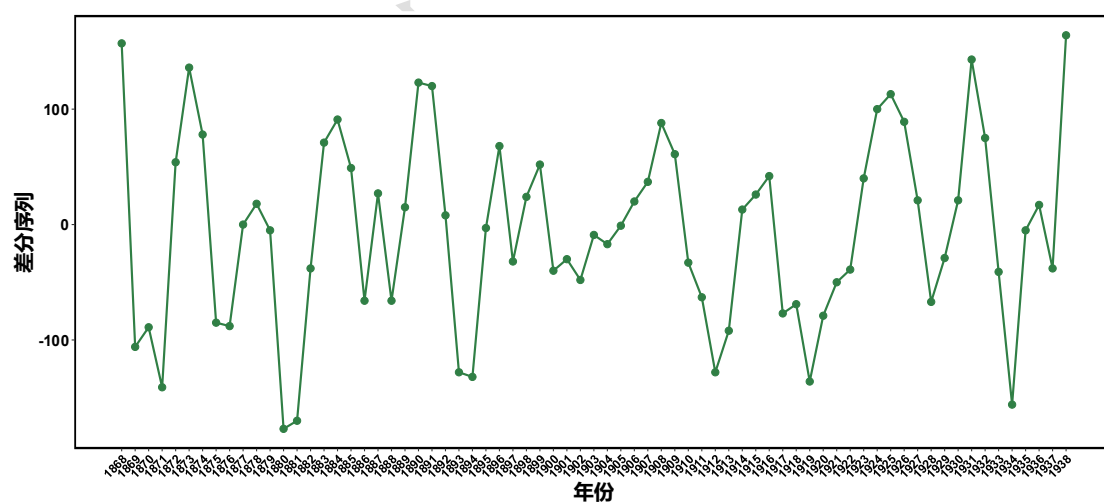


图 10 一阶差分序列图

从一阶差分后的时序图可以看出，差分序列呈现出明显的波动变化。数据点围绕着零值上下波动，显示出序列经过差分处理后可能消除了趋势性。波动模式显示出较高的随机性，没有明显的长期趋势或周期性规律，表明序列可能在去除趋势后变得更加平稳。波动的幅度在不同时间段大致保持一致，显示出方差的稳定性。同时，差分后的序列中没有明显的异常值或极端波动，进一步

支持了序列的平稳性特征。综上所述，一阶差分后的序列在趋势、周期性和方差稳定性方面表现出平稳特征，因此，可以初步判定该序列平稳，但还需进一步借助统计检验方法加以验证。

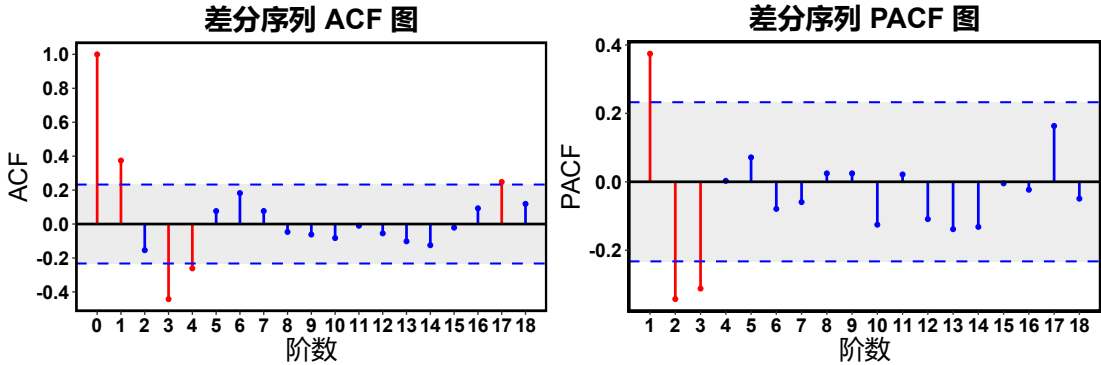


图 11 一阶差分序列的 ACF 图（左）与 PACF 图（右）

从图 11 的特征来看，该序列的平稳性特征较为显著。在 ACF 图中，自相关系数在初始几阶后呈现快速衰减趋势，显示出短期相关性，未能呈现明显的周期性或长期趋势，这一特点与平稳序列的典型特征相吻合。在 PACF 图中，偏自相关系数在延迟阶数 1 之后同样展现出明显的衰减趋势。基于 ACF 图与 PACF 图的观察结果，二者均有力地支持了序列的平稳性。

4. 一阶差分序列的 ADF 检验

为确保结论的严谨性，接下来，本文将结合 ADF 检验方法对一阶差分序列的平稳性进行进一步验证。

由图 11 可知，该序列不具有明显的趋势性；由 ACF 与 PACF 图可知，该序列的自相关系数和偏自相关系数在低阶时表现出显著性，之后迅速衰减并趋于零。考虑到人为判断的主观性，本文选择无趋势情况下的无漂移项自回归结构（类型 1）、有漂移项自回归结构（类型 2）和有趋势情况下的有偏移的回归结构（类型 3）进行延迟 0-4 阶的 ADF 检验。其检验步骤如一、（一）2 所示。

利用 R 语言，针对三种回归模型（类型 1、类型 2 和类型 3）求解 ADF 检验的 P 值，结果如表 10 所示。

表 10 三种自回归类型的 ADF 检验结果（一阶差分序列）

检验类型	延迟阶数	检验统计量 ADF 值	P 值	结论 ($\alpha=0.05$)
无漂移无趋势项	0	-5.3929	$\leq 0.01^{***}$	平稳
无漂移无趋势项	1	-6.6861	$\leq 0.01^{***}$	平稳
无漂移无趋势项	2	-6.5445	$\leq 0.01^{***}$	平稳
无漂移无趋势项	3	-4.7343	$\leq 0.01^{***}$	平稳
无漂移无趋势项	4	-3.7982	$\leq 0.01^{***}$	平稳
有漂移无趋势项	0	-5.3927	$\leq 0.01^{***}$	平稳
有漂移无趋势项	1	-6.6353	$\leq 0.01^{***}$	平稳
有漂移无趋势项	2	-6.5456	$\leq 0.01^{***}$	平稳
有漂移无趋势项	3	-4.7014	$\leq 0.01^{***}$	平稳
有漂移无趋势项	4	-3.8265	$\leq 0.01^{***}$	平稳
有漂移和趋势项	0	-5.4398	$\leq 0.01^{***}$	平稳
有漂移和趋势项	1	-6.6533	$\leq 0.01^{***}$	平稳
有漂移和趋势项	2	-6.5992	$\leq 0.01^{***}$	平稳
有漂移和趋势项	3	-4.7296	$\leq 0.01^{***}$	平稳
有漂移和趋势项	4	-3.9289	0.01796**	平稳

注：*表示显著性 P 值小于 0.10，**表示显著性 P 值小于 0.05，***表示显著性 P 值小于 0.01

由表 10 可知，在显著性水平 $\alpha = 0.05$ 的情况下，一阶差分后的序列在多种检验类型和延迟阶数下均表现出显著的平稳性。具体来看，无漂移无趋势项时，所有延迟阶数的 P 值均小于 0.01；有漂移无趋势项时，所有延迟阶数的 P 值同样小于 0.01；而有漂移和趋势项时，除延迟 4 阶外，其他延迟阶数的 P 值均小于 0.01。因此，可以明确判断该差分后的序列为平稳序列。由于多数检验结果已支持序列的平稳性，无需进一步进行 KPSS 检验。

5. 一阶差分序列的纯随机性检验

本文将对一阶差分序列进行延迟 6 阶、12 阶和 18 阶的纯随机性检验，代入 R 得到结果如表 11 所示。

表 11 一节差分序列的纯随机性结果表

延迟阶数	检验统计量	P 值	结论 ($\alpha=0.05$)
6	35.515	0.000003424***	非白噪声
12	37.348	0.0001961***	非白噪声
18	47.838	0.0001591***	非白噪声

注：*表示显著性 P 值小于 0.10，**表示显著性 P 值小于 0.05，***表示显著性 P 值小于 0.01

由表 11 可知，延迟 6 阶、12 阶和 18 阶的 LB 统计量的 P 值均小于显著性水平 $\alpha = 0.01$ ，所以有理由拒绝该序列为纯随机性的原假设，即该序列为非白噪声序列。

（二）模型拟合、定阶与检验

由二、（一）4、5可知，当考虑三类回归结构时，均可以判断该序列为平稳性序列，并且通过了纯随机性检验，可以进一步采用平稳模型对一阶差分序列进行建模分析。根据的结果知存在三种备选模型。

1. 备选模型 1: AR(3) 模型

由 ACF 图可以看出，自相关系数是以一种有规律按指数函数轨迹递减的，是典型的拖尾特征；由 PACF 图可以看出，除了 3 阶偏自相关系数在 2 倍标准差以外，其他阶数的偏自相关系数均在 2 倍标准差范围内，这是偏自相关系数 3 阶截尾的典型特征，可以将 AR(3) 模型作为备选模型。

2. 备选模型 2: MA(1), MA(4) 模型

由 ACF 图可以看出，除了延迟 1 阶、3 阶、4 阶和 17 阶的自相关系数在 2 倍标准差以外，其他阶数的自相关系数均在 2 倍标准差范围内波动，可以认为该序列自相关系数 2 阶截尾；由 PACF 图可以看出，偏自相关系数显示出拖尾的典型特征，可以将 MA(1) 模型作为备选模型，也可以将 MA(4) 作为备选模型。

3. 备选模型 3: ARMA 系列模型

由 ACF 和 PACF 图可以看出，自相关系数和偏自相关系数均是以一种有规律按指数函数轨迹递减的，是典型的拖尾特征，且都显示出低阶延迟在 2 倍标准差以外，因此，可以将 $ARMA(p, q)$ （其中 p, q 均小于 4）作为备选模型。

4. 模型定阶与选择

本文利用 R 语言对一阶差分序列用 3 种备选模型进行拟合，并采用 CSS-MLE 混合法对模型的参数进行估计，最终得到的关键指标（包括方差估计值、对数似然值、AIC 值及 BIC 值）如表 12 所示。

表 12 各模型关键指标对比表

模型名称	方差估计值	对数似然值	AIC 值	BIC 值
AR(3)	4211	-397.48	802.95	812.00
MA(1)	5181	-404.54	813.08	817.61
MA(4)	4120	-397.21	804.42	815.74
ARMA(1,1)	5137	-404.25	814.50	821.29
ARMA(1,2)	4966	-403.11	814.21	823.27
ARMA(1,3)	4643	-401.74	813.48	824.79
ARMA(1,4)	4062	-397.02	806.05	819.62
ARMA(2,1)	4289	-398.10	804.20	813.25
ARMA(2,2)	4201	-397.41	804.82	816.13
ARMA(2,3)	4185	-397.27	806.55	820.12
ARMA(2,4)	3959	-396.76	807.53	823.37
ARMA(3,1)	4208	-397.45	804.91	816.22
ARMA(3,2)	4191	-397.32	806.64	820.22
ARMA(3,3)	3950	-396.10	806.19	822.03
ARMA(3,4)	3863	-395.51	807.03	825.13
ARMA(4,1)	3906	-395.83	803.67	817.24
ARMA(4,2)	3831	-395.41	804.82	820.66
ARMA(4,3)	4099	-396.69	809.39	827.49
ARMA(4,4)	3861	-395.50	808.99	829.36

由表 12 可知，从方差估计值和对数似然值来看，ARMA(4,2)模型的方差估计最低，表明该模型对数据残差的拟合精度最高；对数似然值最大，进一步印证了其良好的拟合优度。但从 AIC 值与 BIC 值来看，AR(3)模型的 AIC 值和 BIC 值最低，AIC 最低表明其在拟合优度与模型简洁性之间取得了最佳权衡；BIC 最低进一步支持了 AR(3) 作为最简约且有效模型的选择。

基于 AIC 和 BIC 准则，AR(3) 模型是最优备选模型。它在提供极佳拟合效果的同时，保持了高度的简约性。虽然 ARMA(4,2) 在原始拟合优度（方差、似然）上表现最好，但其复杂度的代价过高，导致其信息准则值显著劣于 AR(3)。因此，最终选择 **AR(3) 模型** 作为该一阶差分序列的拟合模型，其形式为

$$\nabla x_t = \phi_0 + \phi_1 \nabla x_{t-1} + \phi_2 \nabla x_{t-2} + \phi_3 \nabla x_{t-3} + \varepsilon_t \quad (10)$$

5. 模型拟合

在选定了 AR(3) 模型后，本文利用 R 语言进行模型的参数估计，得到一阶差分序列的参数估计结果如表 13 所示。

表 13 AR(3) 参数估计表

备选模型	ar1 (标准误差)	ar2 (标准误差)	ar3 (标准误差)
AR(3)	0.5179 (0.1299)	-0.2812 (0.1468)	-0.2794 (0.1298)

由表 13 可知, ϕ_1 估计值为 0.5179, 标准误差 0.1299, 远超两倍标准误差 0.2598, 表明该参数显著。 ϕ_2 估计值为-0.2812, 标准误差 0.1468, 其绝对值略低于两倍标准误差 0.2936, 接近显著水平。 ϕ_3 估计值为-0.2794, 标准误差 0.1298, 超过两倍标准误差 0.2596, 参数显著。总体来看, AR(3) 模型中大部分参数显著, 模型拟合效果较好。

因此, 得到一阶差分序列的拟合模型为:

$$\nabla x_t = 0.5179\nabla x_{t-1} - 0.2812\nabla x_{t-2} - 0.2794\nabla x_{t-3} + \varepsilon_t \quad (11)$$

6. 模型检验

在对各备选模型进行拟合与参数估计后, 本文基于关键指标 (AIC、BIC 等) 选定 AR (3) 模型作为最优模型。然而, 模型选择仅是时间序列分析的一个重要环节, 模型检验同样不可或缺, 它能验证模型的合理性和适用性, 确保模型能准确捕捉序列特征并可靠地进行预测。接下来, 本文将从残差自相关性检验和模型正态性检验两个关键维度出发, 对该模型进行全面诊断。

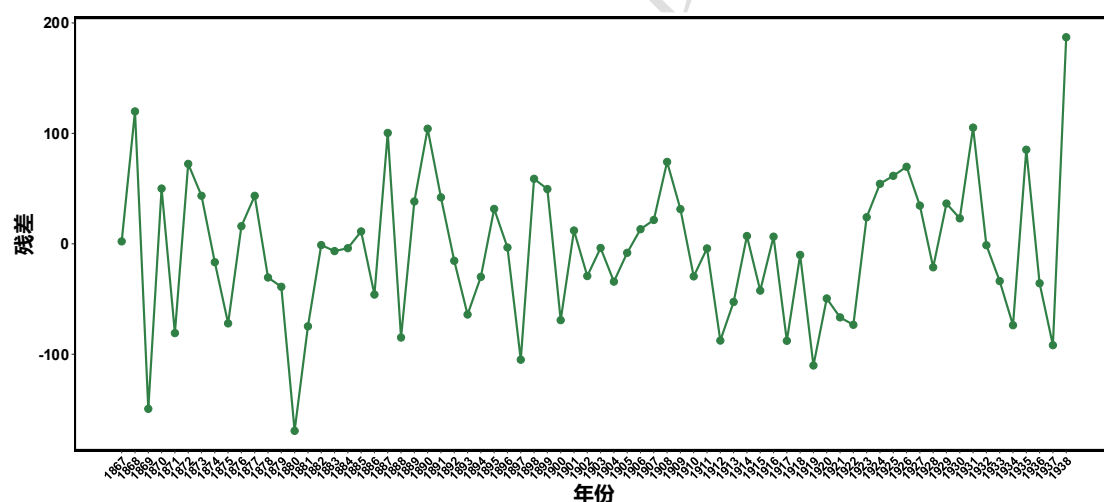


图 12 一阶差分序列的残差图

图 12 显示残差在零附近随机波动, 没有明显的趋势或周期性, 这表明模型已经较好地捕捉了序列中的趋势和季节性成分, 并且残差的波动幅度相对稳定, 没有明显的增大或减小趋势, 这表明模型的方差假设是合理的。

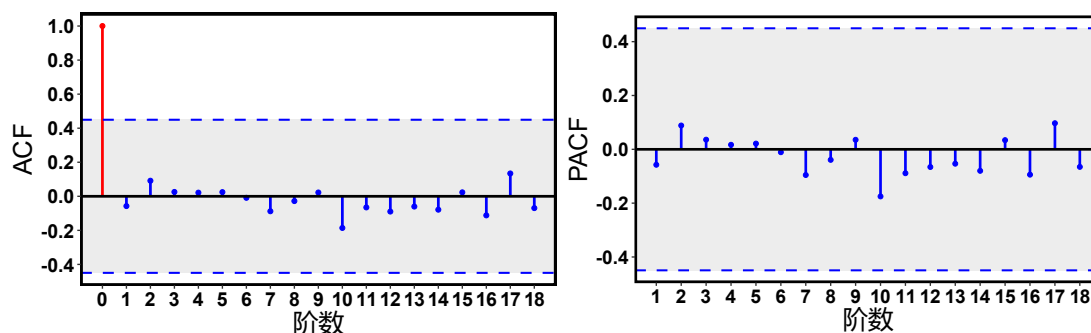


图 13 一阶差分序列残差 ACF 图（左）和 PACF 图（右）

ACF 图显示，除了延迟阶数 0 外，其他延迟阶数的自相关系数都在置信区间内（蓝色虚线之间），这表明残差序列不存在显著的自相关性。这一结果支持了残差序列为白噪声的假设，说明模型已经充分提取了序列中的相关信息。PACF 图同样显示，除了延迟阶数 0 外，其他延迟阶数的偏自相关系数都在置信区间内，进一步支持了残差序列为白噪声的假设。

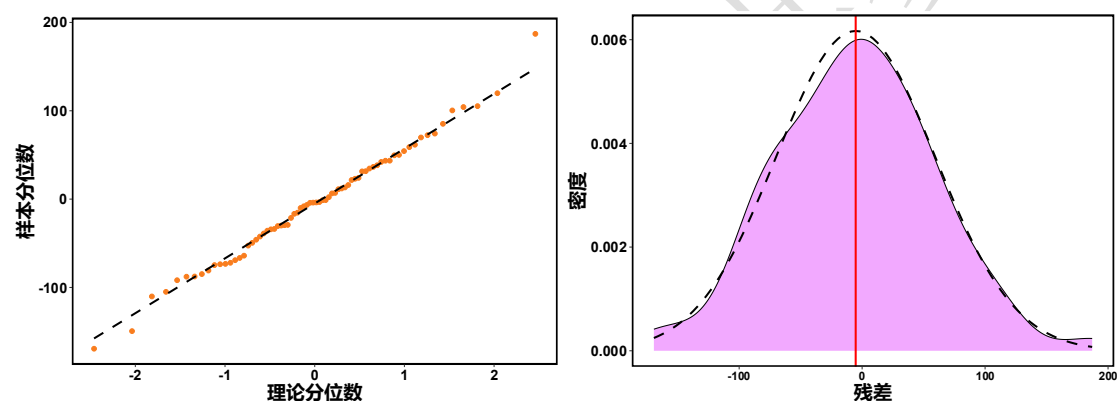


图 14 一阶差分序列残差的 Q-Q 图（左）和密度图（右）

Q-Q 图显示，残差点大致分布在一条直线上，这表明残差的分布接近正态分布。尽管在两端可能存在轻微的偏离，但整体上残差的正态性假设是合理的。密度图显示，残差的密度曲线与正态分布曲线（虚线）基本重合，进一步验证了残差的正态性假设。

在对模型残差进行正态性检验后，为进一步确保模型的适用性与可靠性，本文运用 Ljung-Box 检验对残差序列的自相关性进行严谨评估。Ljung-Box 检验作为一种广泛应用于时间序列分析的统计方法，能够有效检验序列在多个滞后阶数下的自相关性。

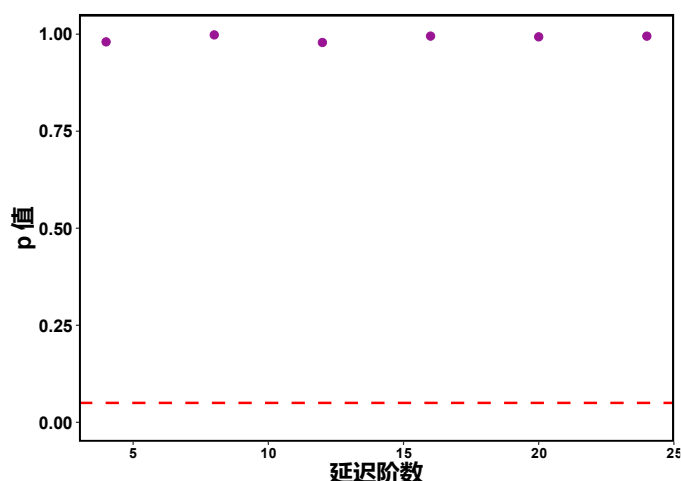


表 14 一阶差分序列的残差序列 P 值显著性图

用 R 语言对 AR (3) 模型的残差进行 Ljung-Box 检验, 结果显示在不同延迟阶数下, 残差序列的 P 值均大于显著性水平 $\alpha = 0.05$ 。这些结果表明, 残差序列可以视为白噪声, 说明 AR (3) 模型充分提取了序列中的相关信息, 拟合效果良好。

7. 参数检验与模型优化

为了进一步验证参数的显著性, 本文采取精确方法 t 检验验算其 P 值, 其步骤如 一、(二) 7 所示。利用 R 语言计算, 得到参数估计显著性结果如 表 15 所示。

表 15 参数估计的显著性水平

参数	t 统计量	P 值
ϕ_1	3.988	0.0000816829***
ϕ_2	1.915	0.0298124315*
ϕ_3	2.153	0.0174161577**

由 表 15 可知, 三个参数的 P 值均小于 0.05, 当 $\alpha = 0.05$ 时拒绝原假设, 即三个参数都显著非零。

因此, 一阶差分序列的拟合模型为

$$\nabla x_t = 0.5179\nabla x_{t-1} - 0.2812\nabla x_{t-2} - 0.2794\nabla x_{t-3} + \varepsilon_t \quad (12)$$

经化简得

$$x_t = 1.5179x_{t-1} - 0.7991x_{t-2} + 0.5606x_{t-3} - 0.2794x_{t-4} + \varepsilon_t \quad (13)$$

8. 拟合预测英国 1867-1938 年的绵羊数量

利用 R 语言对英国 1867-1938 年的绵羊数量进行预测, 得到结果如 图 15 所示。

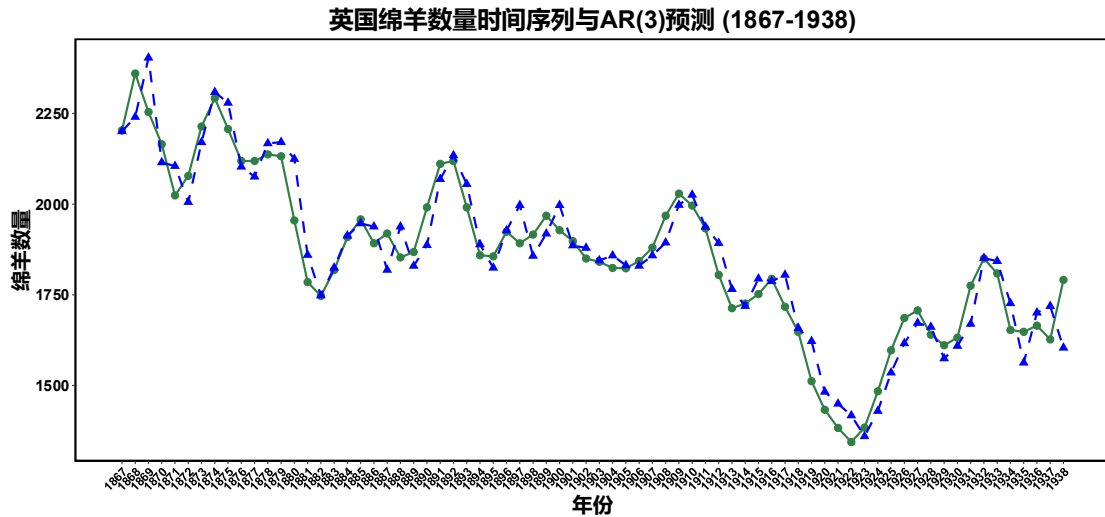


图 15 英国绵羊数量时间序列与预测图

图 15 展示了 1867 年至 1938 年间英国绵羊数量的时间序列数据以及基于 AR(3) 模型的预测值。图中绿色实线和圆点代表实际观测到的绵羊数量数据，蓝色虚线和三角形代表根据 AR(3) 模型拟合出的预测值。从图中可以看出，AR(3) 模型大致捕捉到了绵羊数量的变化趋势，预测值与实际数据的波动较为接近。一些年份预测值与实际值较为接近，显示出模型较好的拟合效果；而在另一些年份则存在一定偏差。总体而言，AR(3) 模型对英国绵羊数量的变化趋势有较好的拟合效果，基本反映了数据的波动情况。

参考文献

- [1] 王燕：《时间序列分析：基于 R》，北京，中国人民大学出版社，2020.

附录

附录 1

介绍：第一道题代码

```
rm(list = ls())
setwd("E:\\6 大三下\\时间序列分析\\期中报告")

# -----

# 加载必要的包
library(ggplot2)
library(reshape2)
library(dplyr)
library(tidyverse)
library(scales)
library(fBasics)
library(fUnitRoots)
library(aTSA)
library(tseries)
library(forecast)

# -----

# 读入数据
data1 <- read.table("data1.txt", header = F)

# 将宽格式数据框转换为长格式
data1 <- data1 %>%
  pivot_longer(cols = everything(), # 转换所有列
               names_to = "年份", # 原列名作为年份列
               values_to = "谷物产量 (千吨)") # 原列的值作为数值列
data1$`年份` <- 1:74

# -----

# 绘制时间序列图
theme_set(theme_bw())
p1 <- ggplot(data1, aes(x = 年份, y = `谷物产量 (千吨)`)) +
  geom_line(color = "#307f45", size = 1) + # 绘制蓝色折线
  geom_point(color = "#307f45", size = 3) + # 绘制蓝色点

# 添加标题和轴标签
labs(
  title = "谷物产量时间序列图",
  x = "年份",
  y = "谷物产量 (千吨)"
) +

# 设置 x 轴刻度
scale_x_continuous(breaks = seq(1, 74, by = 1)) +

# 设置 y 轴刻度
scale_y_continuous(limits = c(0.15, 1.65), breaks = seq(0.15, 1.65, by = 0.05)) +

# 使用简洁的主题
theme(
  legend.position = "none",
  axis.text.x = element_text(angle = 45, hjust = 1, vjust = 1, face = "bold",
  size = 12, color = "black"),
  axis.text.y = element_text(color = "black", size = 15, face = "bold"),
  axis.title.x = element_text(color = "black", size = 20, face = "bold"),
  axis.title.y = element_text(color = "black", size = 20, face = "bold"),
  panel.grid.major = element_blank(),
  panel.grid.minor = element_blank(),
  panel.border = element_rect(color = "black", size = 1.5, linetype = "solid")
)
ggsave("plot1.svg", plot = p1, width = 15, height = 7, dpi = 300)
ggsave("plot1.png", plot = p1, width = 15, height = 7, dpi = 2400)

# -----

# 平稳性检验
# 平稳性检验绘图函数
plot_acf <- function(data, column_name, n_lags = 20, title = NULL) {
  # 计算自相关系数
```

```

acf_result <- acf(data[[column_name]], lag.max = n_lags, plot = FALSE)
acf_values <- data.frame(
  lag = 0:acf_result$lag[n_lags+1],
  acf = acf_result$acf[1:(n_lags+1)]
)

# 计算显著性界限
n <- nrow(data)
sig_bound <- 1.96 * sqrt(1 / n)

# 绘制图形
ggplot(acf_values, aes(x = lag, y = acf)) +
  # 添加显著性区域背景
  geom_rect(aes(ymin = -sig_bound, ymax = sig_bound,
               xmin = -Inf, xmax = Inf),
            fill = "lightblue", alpha = 0.01) +
  # 添加自相关系数线段和点
  geom_segment(aes(xend = lag, yend = 0,
                  color = ifelse(acf > sig_bound |
                                acf < -sig_bound,
                                "significant", "not_significant")),
              size = 1) +
  geom_point(aes(color = ifelse(acf > sig_bound |
                                acf < -sig_bound,
                                "significant", "not_significant")),
             size = 1.5) +
  # 设置颜色映射
  scale_color_manual(values = c("significant" = "red", "not_significant" =
"blue")) +
  # 添加坐标轴标签和标题
  xlab("阶数") +
  ylab("ACF") +
  ggtitle(ifelse(is.null(title), paste(column_name, "ACF 图"), title)) +
  # 设置坐标轴刻度
  scale_x_continuous(breaks = seq(0, n_lags, by = 1)) +
  scale_y_continuous(breaks = seq(-1, 1, by = 0.2),
                    labels = comma) +
  # 设置主题元素
  theme(axis.text.x = element_text(vjust = 1, face = "bold",
                                   size = 15, color = "black"),
        axis.text.y = element_text(color = "black", size = 15,
                                   face = "bold"),
        axis.title.x = element_text(color = "black", size = 20),
        axis.title.y = element_text(color = "black", size = 20),
        panel.grid.major = element_blank(),
        panel.grid.minor = element_blank(),
        panel.border = element_rect(color = "black", size = 1.5,
                                   linetype = "solid"),
        legend.position = "none",
        plot.title = element_text(hjust = 0.5, size = 22, face = "bold")) +
  # 添加参考线
  geom_hline(yintercept = 0, linetype = "solid",
            color = "black", size = 1) +
  geom_hline(yintercept = sig_bound,
            linetype = "dashed", color = "blue", size = 0.75) +
  geom_hline(yintercept = -sig_bound,
            linetype = "dashed", color = "blue", size = 0.75)
}

# 使用示例
p2 <- plot_acf(data1, '谷物产量 (千吨)', 18)
ggsave("plot2.svg", plot = p2, width = 6, height = 4, dpi = 300)
ggsave("plot2.png", plot = p2, width = 6, height = 4, dpi = 2400)

# -----
# 平稳性检验绘图函数 - PACF 图
plot_pacf <- function(data, column_name, n_lags = 20, title = NULL) {
  # 计算偏自相关系数
  pacf_result <- pacf(data[[column_name]], lag.max = n_lags, plot = FALSE)

  # 准备绘图数据, 确保滞后阶数从 1 到 n_lags
  pacf_values <- data.frame(
    lag = 1:n_lags,
    pacf = pacf_result$acf[1:(n_lags)]
  )

  # 计算显著性界限
  n <- nrow(data)
  sig_bound <- 1.96 * sqrt(1 / n)

```

```

# 绘制图形
ggplot(pacf_values, aes(x = lag, y = pacf)) +
  # 添加显著性区域背景
  geom_rect(aes(ymin = -sig_bound, ymax = sig_bound,
               xmin = -Inf, xmax = Inf),
            fill = "lightblue", alpha = 0.01) +
  # 添加偏自相关系数线段和点
  geom_segment(aes(xend = lag, yend = 0,
                  color = ifelse(pacf > sig_bound |
                                pacf < -sig_bound,
                                "significant", "not_significant")),
              size = 1) +
  geom_point(aes(color = ifelse(pacf > sig_bound |
                                pacf < -sig_bound,
                                "significant", "not_significant")),
             size = 1.5) +
  # 设置颜色映射
  scale_color_manual(values = c("significant" = "red", "not_significant" =
"blue")) +
  # 添加坐标轴标签和标题
  xlab("阶数") +
  ylab("PACF") +
  ggtitle(ifelse(is.null(title), paste(column_name, "PACF 图"), title)) +
  # 设置坐标轴刻度
  scale_x_continuous(breaks = seq(1, n_lags, by = 1)) +
  scale_y_continuous(breaks = seq(-1, 1, by = 0.2)) +
  # 设置主题元素
  theme(axis.text.x = element_text(vjust = 1, face = "bold",
                                   size = 15, color = "black"),
        axis.text.y = element_text(color = "black", size = 15,
                                   face = "bold"),
        axis.title.x = element_text(color = "black", size = 20),
        axis.title.y = element_text(color = "black", size = 20),
        panel.grid.major = element_blank(),
        panel.grid.minor = element_blank(),
        panel.border = element_rect(color = "black", size = 1.5,
                                   linetype = "solid"),
        legend.position = "none",
        plot.title = element_text(hjust = 0.5, size = 22, face = "bold")) +
  # 添加参考线
  geom_hline(yintercept = 0, linetype = "solid",
            color = "black", size = 1) +
  geom_hline(yintercept = sig_bound,
            linetype = "dashed", color = "blue", size = 0.75) +
  geom_hline(yintercept = -sig_bound,
            linetype = "dashed", color = "blue", size = 0.75)
}

plot_pacf(data1, '谷物产量 (千吨)', 18)

# 使用示例
p3 <- plot_pacf(data1, '谷物产量 (千吨)', 18)
ggsave("plot3.svg", plot = p3, width = 6, height = 4, dpi = 300)
ggsave("plot3.png", plot = p3, width = 6, height = 4, dpi = 2400)

# -----

## ADF 检验

for(i in 0:2) print(adfTest(data1$`谷物产量 (千吨)` , lag = i, type = "nc"))
for(i in 0:2) print(adfTest(data1$`谷物产量 (千吨)` , lag = i, type = "c"))
for(i in 0:2) print(adfTest(data1$`谷物产量 (千吨)` , lag = i, type = "ct"))
adf.test(data1$`谷物产量 (千吨)`)

# -----

# KPSS 检验
kpss.test(data1$`谷物产量 (千吨)` , null = "Level")
kpss.test(data1$`谷物产量 (千吨)` , null = "Trend")

# -----

# 非随机性检验 (白噪声检验)
Box.test(data1$`谷物产量 (千吨)` , lag = 6, type = "Ljung-Box")
Box.test(data1$`谷物产量 (千吨)` , lag = 12, type = "Ljung-Box")
Box.test(data1$`谷物产量 (千吨)` , lag = 18, type = "Ljung-Box")

```

```

# -----
## 模型拟合

# 手动定阶拟合 AR(1) 模型
AR1_model <- arima(data1$`谷物产量 (千吨)` , order = c(1, 0, 0)) # ARMA(1,1) 模型
print(AR1_model)

# 手动定阶拟合 MA(2) 模型
MA2_model <- arima(data1$`谷物产量 (千吨)` , order = c(0, 0, 2)) # ARMA(1,1) 模型
print(MA2_model)

# 手动定阶拟合 ARMA(1,1) 模型
ARMA11_model <- arima(data1$`谷物产量 (千吨)` , order = c(1, 0, 1)) # ARMA(1,1) 模型
print(ARMA11_model)

# 手动定阶拟合 ARMA(1,2) 模型
ARMA12_model <- arima(data1$`谷物产量 (千吨)` , order = c(1, 0, 2)) # ARMA(1,1) 模型
print(ARMA12_model)

# 手动定阶拟合 ARMA(2,1) 模型
ARMA21_model <- arima(data1$`谷物产量 (千吨)` , order = c(2, 0, 1)) # ARMA(1,1) 模型
print(ARMA21_model)

# 手动定阶拟合 ARMA(2,2) 模型
ARMA22_model <- arima(data1$`谷物产量 (千吨)` , order = c(2, 0, 2)) # ARMA(1,1) 模型
print(ARMA22_model)

# 创建一个数据框来存储模型的 AIC 和 BIC 值
model_comparison <- data.frame(
  Model = c("AR(1)", "MA(2)", "ARMA(1,1)", "ARMA(1,2)", "ARMA(2,1)", "ARMA(2,2)"),
  AIC = c(
    AIC(AR1_model),
    AIC(MA2_model),
    AIC(ARMA11_model),
    AIC(ARMA12_model),
    AIC(ARMA21_model),
    AIC(ARMA22_model)
  ),
  BIC = c(
    BIC(AR1_model),
    BIC(MA2_model),
    BIC(ARMA11_model),
    BIC(ARMA12_model),
    BIC(ARMA21_model),
    BIC(ARMA22_model)
  )
)

# 打印模型比较结果
print(model_comparison)

# 查找 AIC 和 BIC 最小值对应的模型
best_AIC_model <- model_comparison[which.min(model_comparison$AIC), ]
best_BIC_model <- model_comparison[which.min(model_comparison$BIC), ]

print(paste("Best model based on AIC:", best_AIC_model$Model))
print(paste("Best model based on BIC:", best_BIC_model$Model))

# -----
## 模型检验

# 提取模型残差
residuals <- resid(ARMA11_model)

# 计算自相关系数
acf_values <- acf(residuals, plot = FALSE)

# 将自相关系数转换为数据框
acf_df <- data.frame(
  Lag = acf_values$lag[, , 1],
  ACF = acf_values$acf[, , 1]
)

# 绘制残差自相关图
p4 <- plot_acf(acf_df, 'ACF', n_lags = 18)
ggsave("plot4.svg", plot = p4, width = 6, height = 4, dpi = 300)
ggsave("plot4.png", plot = p4, width = 6, height = 4, dpi = 2400)

```



```

# 绘制残差偏自相关图
p5 <- plot_pacf(acf_df, 'ACF', n_lags = 18)
ggsave("plot5.svg", plot = p5, width = 6, height = 4, dpi = 300)
ggsave("plot5.png", plot = p5, width = 6, height = 4, dpi = 2400)

# 将残差和时间信息转换为数据框
residuals_df <- data.frame(
  `年份` = 1:74,
  `残差` = residuals
)

# 绘制残差时间序列图
theme_set(theme_bw())
p6 <- ggplot(residuals_df, aes(x = `年份`, y = `残差`)) +
  geom_line(color = "#307f45", size = 1) + # 绘制蓝色折线
  geom_point(color = "#307f45", size = 3) + # 绘制蓝色点

# 添加标题和轴标签
labs(
  title = "残差序列图",
  x = "年份",
  y = "残差"
) +

# 设置 x 轴刻度
scale_x_continuous(breaks = seq(1, 74, by = 1)) +

# 设置 y 轴刻度
scale_y_continuous(limits = c(-0.5, 0.85), breaks = seq(-0.5, 0.85, by = 0.05),
  labels = scales::number_format()) +

# 使用简洁的主题
theme(
  legend.position = "none",
  axis.text.x = element_text(angle = 45, hjust = 1, vjust = 1, face = "bold",
size = 12, color = "black"),
  axis.text.y = element_text(color = "black", size = 15, face = "bold"),
  axis.title.x = element_text(color = "black", size = 20, face = "bold"),
  axis.title.y = element_text(color = "black", size = 20, face = "bold"),
  panel.grid.major = element_blank(),
  panel.grid.minor = element_blank(),
  panel.border = element_rect(color = "black", size = 1.5, linetype = "solid")
)
ggsave("plot6.svg", plot = p6, width = 15, height = 7, dpi = 300)
ggsave("plot6.png", plot = p6, width = 15, height = 7, dpi = 2400)

# 绘制 Q-Q 图
p7 <- ggplot(residuals_df, aes(sample = `残差`)) +
  stat_qq(color = "#f97d1e", size = 2) + # 设置点的颜色和大小
  stat_qq_line(color = "black", linetype = "dashed", size = 1) + # 添加参考线

# 添加标题和轴标签
labs(
  title = "残差 Q-Q 图",
  x = "理论分位数",
  y = "样本分位数"
) +

# 使用简洁的主题
theme(
  legend.position = "none",
  axis.text.x = element_text(face = "bold", size = 15, color = "black"),
  axis.text.y = element_text(color = "black", size = 15, face = "bold"),
  axis.title.x = element_text(color = "black", size = 20, face = "bold"),
  axis.title.y = element_text(color = "black", size = 20, face = "bold"),
  panel.grid.major = element_blank(),
  panel.grid.minor = element_blank(),
  panel.border = element_rect(color = "black", size = 1.5, linetype = "solid")
)
ggsave("plot7.svg", plot = p7, width = 8, height = 6, dpi = 300)
ggsave("plot7.png", plot = p7, width = 8, height = 6, dpi = 2400)

# 残差密度图
p8 <- ggplot(residuals_df, aes(x = `残差`)) +
  geom_density(fill = "#ed85ff", alpha = 0.7) +
  stat_function(
    fun = dnorm,

```

```

    args = list(mean = mean(residuals), sd = sd(residuals)),
    color = "black",
    linetype = "dashed",
    size = 1
  ) +
  geom_vline(
    xintercept = mean(residuals),
    color = "red",
    size = 1
  ) +
  labs(title = "残差密度图", x = "残差", y = "密度") +
  scale_y_continuous(labels = scales::number_format()) + # 避免科学计数法
  theme(
    legend.position = "none",
    axis.text.x = element_text(face = "bold", size = 12, color = "black"),
    axis.text.y = element_text(color = "black", size = 15, face = "bold"),
    axis.title.x = element_text(color = "black", size = 20, face = "bold"),
    axis.title.y = element_text(color = "black", size = 20, face = "bold"),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    panel.border = element_rect(color = "black", size = 1.5, linetype = "solid")
  )
ggsave("plot8.svg", plot = p8, width = 8, height = 6, dpi = 300)
ggsave("plot8.png", plot = p8, width = 8, height = 6, dpi = 2400)

t <- ts.diag(ARMA11_model)
# 将 t 转换为数据框
lb_result <- as.data.frame(t)

# 绘制 p 值图
theme_set(theme_bw())
p9 <- ggplot(lb_result, aes(x = lag, y = p.value)) +
  geom_hline(yintercept = 0.05, color = "red", linetype = "dashed", size = 1) +
  geom_point(color = "#9a1593", size = 3) +
  scale_y_continuous(limits = c(0, 1)) +
  labs(title = "Ljung-Box 检验 p 值图", x = "延迟阶数", y = "p 值") +
  theme(
    axis.text.x = element_text(face = "bold", size = 12, color = "black"),
    axis.text.y = element_text(face = "bold", size = 15, color = "black"),
    axis.title.x = element_text(face = "bold", size = 20, color = "black"),
    axis.title.y = element_text(face = "bold", size = 20, color = "black"),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    panel.border = element_rect(color = "black", size = 1.5, linetype = "solid")
  )
ggsave("plot9.svg", plot = p9, width = 8, height = 6, dpi = 300)
ggsave("plot9.png", plot = p9, width = 8, height = 6, dpi = 2400)

# 使用 Shapiro-wilk 检验进行正态性检验
shapiro_test <- shapiro.test(residuals)
print(shapiro_test)

# 使用 Ljung-Box 检验对残差进行纯随机性检验
checkresiduals(ARMA11_model)

# -----

## 参数检验

# 构造 t 统计量
t <- abs(ARMA11_model$coef)/sqrt(diag(ARMA11_model$var.coef))
t

# 用 pt 函数计算 p 值
pt(t, length(data1$`谷物产量(千吨)`)) - length(ARMA11_model$coef), lower.tail = F)

# -----

## 模型拟合

# 生成未来 5 年的预测 (包括置信区间)
forecast_ARMA11 <- forecast(ARMA11_model, h = 5, level = c(80, 95)) # 80%和 95%置信
区间

# 将预测结果转换为数据框
forecast_df <- with(forecast_ARMA11, {
  data.frame(
    年份 = 75:79,

```

```

    预测值 = as.numeric(mean),
    低 80 = as.numeric(lower[,1]),
    高 80 = as.numeric(upper[,1]),
    低 95 = as.numeric(lower[,2]),
    高 95 = as.numeric(upper[,2])
  )
})

# 绘制完整图表 (历史数据 + 预测 + 置信区间)
theme_set(theme_bw())
p9 <- ggplot() +
  # 绘制历史数据
  geom_line(data = data1, aes(x = 年份, y = `谷物产量 (千吨)`),
            color = "#307f45", size = 1) +
  geom_point(data = data1, aes(x = 年份, y = `谷物产量 (千吨)`),
             color = "#307f45", size = 3) +

  # 绘制预测数据
  geom_line(data = forecast_df, aes(x = 年份, y = 预测值),
            color = "blue", size = 1) +
  geom_point(data = forecast_df, aes(x = 年份, y = 预测值),
             color = "blue", size = 3) +

  # 绘制置信区间 (95% 和 80%)
  geom_ribbon(data = forecast_df, aes(x = 年份, ymin = 低 95, ymax = 高 95),
             fill = "#984ea3", alpha = 0.1) +
  geom_ribbon(data = forecast_df, aes(x = 年份, ymin = 低 80, ymax = 高 80),
             fill = "#e41a1c", alpha = 0.15) +

  # 添加标题和轴标签
  labs(
    title = "谷物产量时间序列与 ARMA(1,1) 预测",
    x = "年份",
    y = "谷物产量 (千吨)"
  ) +

  # 设置 x 轴刻度 (包含预测年份)
  scale_x_continuous(breaks = seq(1, 79, by = 2)) + # 每 5 年显示一个刻度

  # 设置 y 轴刻度
  scale_y_continuous(limits = c(0, 1.75), breaks = seq(0, 1.75, by = 0.25)) +

  # 使用简洁的主题
  theme(
    legend.position = "none",
    axis.text.x = element_text(angle = 45, hjust = 1, vjust = 1, face = "bold",
                                size = 12, color = "black"),
    axis.text.y = element_text(color = "black", size = 15, face = "bold"),
    axis.title.x = element_text(color = "black", size = 20, face = "bold"),
    axis.title.y = element_text(color = "black", size = 20, face = "bold"),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    panel.border = element_rect(color = "black", size = 1.5, linetype = "solid"),
    plot.title = element_text(size = 24, face = "bold", hjust = 0.5)
  )
ggsave("plot9.svg", plot = p9, width = 15, height = 7, dpi = 300)
ggsave("plot9.png", plot = p9, width = 15, height = 7, dpi = 2400)

```

附录 2

介绍：第二道题代码

```

rm(list = ls())
setwd("E:\\6 大三下\\时间序列分析\\期中报告")

# -----

# 加载必要的包
library(ggplot2)
library(reshape2)
library(dplyr)
library(tidyverse)
library(scales)
library(FBasics)
library(fUnitRoots)
library(aTSA)
library(tseries)
library(forecast)

```

```

# -----

# 读入数据
data2 <- read.table("data2.txt", header = F)

# 将宽格式数据框转换为长格式
data2 <- data2 %>%
  pivot_longer(cols = everything(), # 转换所有列
               names_to = "年份", # 原列名作为年份列
               values_to = "绵羊数量") # 原列的值作为数值列
data2$`年份` <- 1867:1938

# -----

# 设置主题
theme_set(theme_bw())
# 绘制时间序列图
p10 <- ggplot(data2, aes(x = 年份, y = 绵羊数量)) +
  geom_line(color = "#307f45", size = 1) + # 绿色折线
  geom_point(color = "#307f45", size = 3) + # 绿色点

# 添加标题和轴标签
labs(
  title = "绵羊数量时间序列图",
  x = "年份",
  y = "绵羊数量"
) +

# 设置 x 轴刻度 (根据年份范围自动调整)
scale_x_continuous(breaks = seq(1867, 1938, by = 1)) +

# 设置 y 轴刻度 (根据数据范围调整)
scale_y_continuous(limits = c(1300, 2400), breaks = seq(1300, 2400, by = 100)) +

# 使用简洁的主题
theme(
  legend.position = "none",
  axis.text.x = element_text(angle = 45, hjust = 1, vjust = 1, face = "bold",
size = 12, color = "black"),
  axis.text.y = element_text(color = "black", size = 15, face = "bold"),
  axis.title.x = element_text(color = "black", size = 20, face = "bold"),
  axis.title.y = element_text(color = "black", size = 20, face = "bold"),
  panel.grid.major = element_blank(),
  panel.grid.minor = element_blank(),
  panel.border = element_rect(color = "black", size = 1.5, linetype = "solid")
)

# 保存图片
ggsave("plot10.svg", plot = p10, width = 15, height = 7, dpi = 300)
ggsave("plot10.png", plot = p10, width = 15, height = 7, dpi = 2400)

# -----

# 平稳性检验
# 平稳性检验绘图函数
plot_acf <- function(data, column_name, n_lags = 20, title = NULL) {
  # 计算自相关系数
  acf_result <- acf(data[[column_name]], lag.max = n_lags, plot = FALSE)
  acf_values <- data.frame(
    lag = 0:acf_result$lag[n_lags+1],
    acf = acf_result$acf[1:(n_lags+1)]
  )

  # 计算显著性界限
  n <- nrow(data)
  sig_bound <- 1.96 * sqrt(1 / n)

  # 绘制图形
  ggplot(acf_values, aes(x = lag, y = acf)) +
    # 添加显著性区域背景
    geom_rect(aes(ymin = -sig_bound, ymax = sig_bound,
                  xmin = -Inf, xmax = Inf),
              fill = "lightblue", alpha = 0.01) +
    # 添加自相关系数线段和点
    geom_segment(aes(xend = lag, yend = 0,
                     color = ifelse(acf > sig_bound |

```

```

                                acf <- -sig_bound,
                                "significant", "not_significant")),
                                size = 1) +
geom_point(aes(color = ifelse(acf > sig_bound |
                                acf < -sig_bound,
                                "significant", "not_significant")),
                                size = 1.5) +
# 设置颜色映射
scale_color_manual(values = c("significant" = "red", "not_significant" =
"blue")) +
# 添加坐标轴标签和标题
xlab("阶数") +
ylab("ACF") +
ggtitle(ifelse(is.null(title), paste(column_name, "ACF 图"), title)) +
# 设置坐标轴刻度
scale_x_continuous(breaks = seq(0, n_lags, by = 1)) +
scale_y_continuous(breaks = seq(-1, 1, by = 0.2),
                    labels = comma) +
# 设置主题元素
theme(axis.text.x = element_text(vjust = 1, face = "bold",
                                size = 15, color = "black"),
      axis.text.y = element_text(color = "black", size = 15,
                                face = "bold"),
      axis.title.x = element_text(color = "black", size = 20),
      axis.title.y = element_text(color = "black", size = 20),
      panel.grid.major = element_blank(),
      panel.grid.minor = element_blank(),
      panel.border = element_rect(color = "black", size = 1.5,
                                linetype = "solid"),
      legend.position = "none",
      plot.title = element_text(hjust = 0.5, size = 22, face = "bold")) +
# 添加参考线
geom_hline(yintercept = 0, linetype = "solid",
           color = "black", size = 1) +
geom_hline(yintercept = sig_bound,
           linetype = "dashed", color = "blue", size = 0.75) +
geom_hline(yintercept = -sig_bound,
           linetype = "dashed", color = "blue", size = 0.75)
}

# 使用示例
p11 <- plot_acf(data2, '绵羊数量', 18)
ggsave("plot11.svg", plot = p11, width = 6, height = 4, dpi = 300)
ggsave("plot11.png", plot = p11, width = 6, height = 4, dpi = 2400)

# -----

# 平稳性检验绘图函数 - PACF 图
plot_pacf <- function(data, column_name, n_lags = 20, title = NULL) {
# 计算偏自相关系数
pacf_result <- pacf(data[[column_name]], lag.max = n_lags, plot = FALSE)

# 准备绘图数据, 确保滞后阶数从 1 到 n_lags
pacf_values <- data.frame(
  lag = 1:n_lags,
  pacf = pacf_result$acf[1:(n_lags)]
)

# 计算显著性界限
n <- nrow(data)
sig_bound <- 1.96 * sqrt(1 / n)

# 绘制图形
ggplot(pacf_values, aes(x = lag, y = pacf)) +
# 添加显著性区域背景
geom_rect(aes(ymin = -sig_bound, ymax = sig_bound,
              xmin = -Inf, xmax = Inf),
          fill = "lightblue", alpha = 0.01) +
# 添加偏自相关系数线段和点
geom_segment(aes(xend = lag, yend = 0,
                 color = ifelse(pacf > sig_bound |
                               pacf < -sig_bound,
                               "significant", "not_significant")),
             size = 1) +
geom_point(aes(color = ifelse(pacf > sig_bound |
                              pacf < -sig_bound,
                              "significant", "not_significant")),
            size = 1.5) +
# 设置颜色映射
scale_color_manual(values = c("significant" = "red", "not_significant" =
"blue")) +

```

```

# 添加坐标轴标签和标题
xlab("阶数") +
ylab("PACF") +
ggtitle(ifelse(is.null(title), paste(column_name, "PACF 图"), title)) +
# 设置坐标轴刻度
scale_x_continuous(breaks = seq(1, n_lags, by = 1)) +
scale_y_continuous(breaks = seq(-1, 1, by = 0.2)) +
# 设置主题元素
theme(axis.text.x = element_text(vjust = 1, face = "bold",
                                  size = 15, color = "black"),
      axis.text.y = element_text(color = "black", size = 15,
                                  face = "bold"),
      axis.title.x = element_text(color = "black", size = 20),
      axis.title.y = element_text(color = "black", size = 20),
      panel.grid.major = element_blank(),
      panel.grid.minor = element_blank(),
      panel.border = element_rect(color = "black", size = 1.5,
                                  linetype = "solid"),
      legend.position = "none",
      plot.title = element_text(hjust = 0.5, size = 22, face = "bold")) +
# 添加参考线
geom_hline(yintercept = 0, linetype = "solid",
           color = "black", size = 1) +
geom_hline(yintercept = sig_bound,
           linetype = "dashed", color = "blue", size = 0.75) +
geom_hline(yintercept = -sig_bound,
           linetype = "dashed", color = "blue", size = 0.75)
}

# 使用示例
p12 <- plot_pacf(data2, '绵羊数量', 18)
ggsave("plot12.svg", plot = p12, width = 6, height = 4, dpi = 300)
ggsave("plot12.png", plot = p12, width = 6, height = 4, dpi = 2400)

# -----

## ADF 检验

for(i in 0:2) print(adfTest(data2$`绵羊数量`, lag = i, type = "nc"))
for(i in 0:2) print(adfTest(data2$`绵羊数量`, lag = i, type = "c"))
for(i in 0:2) print(adfTest(data2$`绵羊数量`, lag = i, type = "ct"))
adf.test(data2$`绵羊数量`)

# -----

# 差分运算

data2_ts <- ts(data2$`绵羊数量`, start = 1867)
dif_data2 <- data.frame(`年份` = 1868:1938)
dif_data2$`差分序列` <- diff(data2$`绵羊数量`)

# 设置主题
theme_set(theme_bw())
# 绘制时间序列图
p12 <- ggplot(dif_data2, aes(x = 年份, y = 差分序列)) +
  geom_line(color = "#307f45", size = 1) + # 绿色折线
  geom_point(color = "#307f45", size = 3) + # 绿色点

# 添加标题和轴标签
labs(
  title = "一阶差分时间序列图",
  x = "年份",
  y = "差分序列"
) +

# 设置 x 轴刻度 (根据年份范围自动调整)
scale_x_continuous(breaks = seq(1868, 1938, by = 1)) +

# 使用简洁的主题
theme(
  legend.position = "none",
  axis.text.x = element_text(angle = 45, hjust = 1, vjust = 1, face = "bold",
                              size = 12, color = "black"),
  axis.text.y = element_text(color = "black", size = 15, face = "bold"),
  axis.title.x = element_text(color = "black", size = 20, face = "bold"),
  axis.title.y = element_text(color = "black", size = 20, face = "bold"),

```

```

    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    panel.border = element_rect(color = "black", size = 1.5, linetype = "solid")
  )
p12

# 保存图片
ggsave("plot12.svg", plot = p12, width = 15, height = 7, dpi = 300)
ggsave("plot12.png", plot = p12, width = 15, height = 7, dpi = 2400)

# -----

# 差分序列的 ACF 与 PACF 图

p13 <- plot_acf(dif_data2, '差分序列', 18)
ggsave("plot13.svg", plot = p13, width = 6, height = 4, dpi = 300)
ggsave("plot13.png", plot = p13, width = 6, height = 4, dpi = 2400)

p14 <- plot_pacf(dif_data2, '差分序列', 18)
ggsave("plot14.svg", plot = p14, width = 6, height = 4, dpi = 300)
ggsave("plot14.png", plot = p14, width = 6, height = 4, dpi = 2400)

# -----

## ADF 检验

for(i in 0:4) print(adfTest(dif_data2$`差分序列`, lag = i, type = "nc"))
for(i in 0:4) print(adfTest(dif_data2$`差分序列`, lag = i, type = "c"))
for(i in 0:4) print(adfTest(dif_data2$`差分序列`, lag = i, type = "ct"))
adf.test(dif_data2$`差分序列`)

# -----

## 纯随机性检验

# 非随机性检验（白噪声检验）
Box.test(dif_data2$`差分序列`, lag = 6, type = "Ljung-Box")
Box.test(dif_data2$`差分序列`, lag = 12, type = "Ljung-Box")
Box.test(dif_data2$`差分序列`, lag = 18, type = "Ljung-Box")

# -----

## 模型拟合

# 手动定阶拟合 AR(3) 模型
AR3_model <- arima(data2$`绵羊数量`, order = c(3, 1, 0)) # AR(3) 模型
print(AR3_model)

# 手动定阶拟合 MA(1) 模型
MA1_model <- arima(data2$`绵羊数量`, order = c(0, 1, 1)) # MA(1) 模型
print(MA1_model)

# 手动定阶拟合 MA(4) 模型
MA4_model <- arima(data2$`绵羊数量`, order = c(0, 1, 4)) # MA(4) 模型
print(MA4_model)

# 创建一个列表来存储模型结果
arma_models <- list()

# 循环遍历 p 和 q 的值（从 1 到 4）
for (p in 1:4) {
  for (q in 1:4) {
    # 拟合 ARIMA(p, d, q) 模型，这里 d 假设为 1，可根据实际情况调整
    current_model <- arima(data2$`绵羊数量`, order = c(p, 1, q))
    # 将模型存储在列表中，以 p 和 q 为标识
    arma_models[[paste("ARIMA", p, "1", q)]] <- current_model
  }
}

# 查看模型列表
arma_models

# 创建一个数据框来存储模型的 AIC 和 BIC 值
model_comparison <- data.frame(

```

```

    Model = character(),
    AIC = numeric(),
    BIC = numeric()
  )

# 提取已手动拟合模型的 AIC 和 BIC 值
arma_model_list <- list(AR3_model, MA1_model, MA4_model)
arma_model_names <- c("AR(3)", "MA(1)", "MA(4)")

for (i in seq_along(arma_model_list)) {
  model_comparison <- rbind(model_comparison, data.frame(
    Model = arma_model_names[i],
    AIC = AIC(arma_model_list[[i]]),
    BIC = BIC(arma_model_list[[i]])
  ))
}

# 提取循环拟合模型的 AIC 和 BIC 值
for (name in names(arma_models)) {
  model_comparison <- rbind(model_comparison, data.frame(
    Model = name,
    AIC = AIC(arma_models[[name]]),
    BIC = BIC(arma_models[[name]])
  ))
}

# 打印模型比较结果
print(model_comparison)

# 查找 AIC 和 BIC 最小值对应的模型
best_AIC_model <- model_comparison[which.min(model_comparison$AIC), ]
best_BIC_model <- model_comparison[which.min(model_comparison$BIC), ]

print(paste("Best model based on AIC:", best_AIC_model$Model))
print(paste("Best model based on BIC:", best_BIC_model$Model))

# -----

## 模型检验

# 提取模型残差
residuals <- resid(AR3_model)

# 计算自相关系数
acf_values <- acf(residuals, plot = FALSE)

# 将自相关系数转换为数据框
acf_df <- data.frame(
  Lag = acf_values$lag[, , 1],
  ACF = acf_values$acf[, , 1]
)

# 绘制残差自相关图
p15 <- plot_acf(acf_df, 'ACF', n_lags = 18)
ggsave("plot15.svg", plot = p15, width = 6, height = 4, dpi = 300)
ggsave("plot15.png", plot = p15, width = 6, height = 4, dpi = 2400)

# 绘制残差偏自相关图
p16 <- plot_pacf(acf_df, 'ACF', n_lags = 18)
ggsave("plot16.svg", plot = p16, width = 6, height = 4, dpi = 300)
ggsave("plot16.png", plot = p16, width = 6, height = 4, dpi = 2400)

# 将残差和时间信息转换为数据框
residuals_df <- data.frame(
  `年份` = 1867:1938,
  `残差` = residuals
)

# 绘制残差时间序列图
theme_set(theme_bw())
p17 <- ggplot(residuals_df, aes(x = `年份`, y = `残差`)) +
  geom_line(color = "#307f45", size = 1) + # 绘制蓝色折线
  geom_point(color = "#307f45", size = 3) + # 绘制蓝色点

# 添加标题和轴标签
labs(
  title = "残差序列图",
  x = "年份",

```



```

) +

# 设置 x 轴刻度
scale_x_continuous(breaks = seq(1867, 1938, by = 1)) +

# 设置 y 轴刻度
#scale_y_continuous(limits = c(-0.5, 0.85), breaks = seq(-0.5, 0.85, by = 0.05),
#                    labels = scales::number_format()) +

# 使用简洁的主题
theme(
  legend.position = "none",
  axis.text.x = element_text(angle = 45, hjust = 1, vjust = 1, face = "bold",
size = 12, color = "black"),
  axis.text.y = element_text(color = "black", size = 15, face = "bold"),
  axis.title.x = element_text(color = "black", size = 20, face = "bold"),
  axis.title.y = element_text(color = "black", size = 20, face = "bold"),
  panel.grid.major = element_blank(),
  panel.grid.minor = element_blank(),
  panel.border = element_rect(color = "black", size = 1.5, linetype = "solid")
)
ggsave("plot17.svg", plot = p17, width = 15, height = 7, dpi = 300)
ggsave("plot17.png", plot = p17, width = 15, height = 7, dpi = 2400)

# 绘制 Q-Q 图
p18 <- ggplot(residuals_df, aes(sample = `残差`)) +
  stat_qq(color = "#f9d1e", size = 2) + # 设置点的颜色和大小
  stat_qq_line(color = "black", linetype = "dashed", size = 1) + # 添加参考线

# 添加标题和轴标签
labs(
  title = "残差 Q-Q 图",
  x = "理论分位数",
  y = "样本分位数"
) +

# 使用简洁的主题
theme(
  legend.position = "none",
  axis.text.x = element_text(face = "bold", size = 15, color = "black"),
  axis.text.y = element_text(color = "black", size = 15, face = "bold"),
  axis.title.x = element_text(color = "black", size = 20, face = "bold"),
  axis.title.y = element_text(color = "black", size = 20, face = "bold"),
  panel.grid.major = element_blank(),
  panel.grid.minor = element_blank(),
  panel.border = element_rect(color = "black", size = 1.5, linetype = "solid")
)
ggsave("plot18.svg", plot = p18, width = 8, height = 6, dpi = 300)
ggsave("plot18.png", plot = p18, width = 8, height = 6, dpi = 2400)

# 残差密度图
p19 <- ggplot(residuals_df, aes(x = `残差`)) +
  geom_density(fill = "#ed85ff", alpha = 0.7) +
  stat_function(
    fun = dnorm,
    args = list(mean = mean(residuals), sd = sd(residuals)),
    color = "black",
    linetype = "dashed",
    size = 1
  ) +
  geom_vline(
    xintercept = mean(residuals),
    color = "red",
    size = 1
  ) +
  labs(title = "残差密度图", x = "残差", y = "密度") +
  scale_y_continuous(labels = scales::number_format()) + # 避免科学计数法
  theme(
    legend.position = "none",
    axis.text.x = element_text(face = "bold", size = 12, color = "black"),
    axis.text.y = element_text(color = "black", size = 15, face = "bold"),
    axis.title.x = element_text(color = "black", size = 20, face = "bold"),
    axis.title.y = element_text(color = "black", size = 20, face = "bold"),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    panel.border = element_rect(color = "black", size = 1.5, linetype = "solid")
  )
ggsave("plot19.svg", plot = p19, width = 8, height = 6, dpi = 300)
ggsave("plot19.png", plot = p19, width = 8, height = 6, dpi = 2400)

```

```

t <- ts.diag(AR3_model)
# 将 t 转换为数据框
lb_result <- as.data.frame(t)

# 绘制 p 值图
theme_set(theme_bw())
p20 <- ggplot(lb_result, aes(x = lag, y = p.value)) +
  geom_hline(yintercept = 0.05, color = "red", linetype = "dashed", size = 1) +
  geom_point(color = "#9a1593", size = 3) +
  scale_y_continuous(limits = c(0, 1)) +
  labs(title = "Ljung-Box 检验 p 值图", x = "延迟阶数", y = "p 值") +
  theme(
    axis.text.x = element_text(face = "bold", size = 12, color = "black"),
    axis.text.y = element_text(face = "bold", size = 15, color = "black"),
    axis.title.x = element_text(face = "bold", size = 20, color = "black"),
    axis.title.y = element_text(face = "bold", size = 20, color = "black"),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    panel.border = element_rect(color = "black", size = 1.5, linetype = "solid")
  )
ggsave("plot20.svg", plot = p20, width = 8, height = 6, dpi = 300)
ggsave("plot20.png", plot = p20, width = 8, height = 6, dpi = 2400)

# 使用 Shapiro-wilk 检验进行正态性检验
shapiro_test <- shapiro.test(residuals)
print(shapiro_test)

# 使用 Ljung-Box 检验对残差进行纯随机性检验
checkresiduals(AR3_model)

# -----

## 参数检验

# 构造 t 统计量
t <- abs(AR3_model$coef)/sqrt(diag(AR3_model$var.coef))
t

# 用 pt 函数计算 p 值
pt(t, length(data2$`绵羊数量`) - length(AR3_model$coef), lower.tail = F)

# -----

## 模型预测

# 将数据转换为时间序列对象
data2_ts <- ts(data2$绵羊数量, start = 1867, frequency = 1)

# 拟合 AR(3)模型
AR3_model <- arima(data2_ts, order = c(3, 1, 0))

# 生成整个时间段的预测 (1867-1938)
forecast_AR3 <- forecast(AR3_model, h = length(data2_ts))

# 将预测结果转换为数据框
forecast_df <- data.frame(
  年份 = time(data2_ts)[length(data2_ts) - length(forecast_AR3$fitted) +
1:length(forecast_AR3$fitted)],
  预测值 = forecast_AR3$fitted
)

# 创建原始数据的数据框
data2_df <- data.frame(
  年份 = time(data2_ts),
  绵羊数量 = data2_ts
)

# 绘制完整图表 (历史数据 + 预测)
theme_set(theme_bw())
p21 <- ggplot() +
  # 绘制历史数据
  geom_line(data = data2_df, aes(x = 年份, y = 绵羊数量),
    color = "#307f45", size = 1) +
  geom_point(data = data2_df, aes(x = 年份, y = 绵羊数量),
    color = "#307f45", size = 3) +

```

```

# 绘制预测数据
geom_line(data = forecast_df, aes(x = 年份, y = 预测值),
          color = "blue", size = 1, linetype = "dashed") +
geom_point(data = forecast_df, aes(x = 年份, y = 预测值),
           color = "blue", size = 3, shape = 17) +

# 添加标题和轴标签
labs(
  title = "英国绵羊数量时间序列与 AR(3) 预测 (1867-1938)",
  x = "年份",
  y = "绵羊数量"
) +

# 设置 x 轴刻度 (包含所有年份)
scale_x_continuous(breaks = seq(1867, 1938, by = 1)) + # 每 5 年显示一个刻度

# 使用简洁的主题
theme(
  legend.position = "none",
  axis.text.x = element_text(angle = 45, hjust = 1, vjust = 1, face = "bold",
size = 12, color = "black"),
  axis.text.y = element_text(color = "black", size = 15, face = "bold"),
  axis.title.x = element_text(color = "black", size = 20, face = "bold"),
  axis.title.y = element_text(color = "black", size = 20, face = "bold"),
  panel.grid.major = element_blank(),
  panel.grid.minor = element_blank(),
  panel.border = element_rect(color = "black", size = 1.5, linetype = "solid"),
  plot.title = element_text(size = 24, face = "bold", hjust = 0.5)
)

# 保存图像
ggsave("plot21.svg", plot = p21, width = 15, height = 7, dpi = 300)
ggsave("plot21.png", plot = p21, width = 15, height = 7, dpi = 2400)

```